Tutorial

InSAR Data Processing for Urban City Deformation using Open Source Computing Environment



Prepared By

Dr. Hari Shankar
Scientist/Engineer 'SF'

Disaster Management Studies Department
Indian Institute of Remote Sensing (IIRS)
Indian Space Research Organization (ISRO)
Department of Space, Government of India
Dehradun, Uttarakhand, India

INDIAN INSTITUTE OF REMPOTE SENSING

REPORT/ DOUCUMENT CONTROL SHEET

1.	**Security Classification**	Unrestricted	
2.	**Distribution**	Course Participants of Summer Intership at India Space Lab	
3.	**Report Document Version**	**(a) Issue no. 1**	**(b) Revision & Date** May 2026
4.	**Report Document Type**	Practical Tutorial	
5.	**Document Control Number**	IIRS/DMSD/Training/378/2025	
6.	**Title**	InSAR Data Processing for Urban Deformation using MintPy Package	
7.	**Particulars of Collation**	**Page** 68 **Figures** 45 **Tables** 3 **Reference** 9	
8.	**Author(s)**	Dr. Hari Shankar	
9.	**Affiliation of Authors**	DMSD / G&DMG / IIRS / ISRO Dehradun, Uttarakhand, India	
10.	**Security Mechanism**	**Compiled by/ Controlled by** Dr. Hari Shankar Scientist, DMSD	
11.	**Originating Unit**	IIRS Dehradun, Uttarakhand, India	
12.	**Sponsor(s)/ Name and Address**	IIRS Dehradun, Uttarakhand, India	
13.	**Date of Initiation**	April 2026	
14.	**Date of Publication**	May 2026	
15.	**Abstract (with keywords)**	This tutorial deals with the Interferometric Synthetic Aperture Radar (InSAR) data processing for urban land deformation time-series analysis using open source computing environment (MintPy Package, ISCE and Hyp3 SAR Processors). Multi-temporal Small Baseline Subset (SBAS) InSAR techniques is used for deriving time-series land surface movement. Higher order derivatives (velocity and acceleration) of displacement are also calculated from time-series displacement of national capital of India i.e. Delhi City. **Keywords:** Multi-temporal InSAR, Phase time-series, Phase correction, Unwrapping, Inversion, Velocity, Acceleration, Time-Series Visualization.	
16.	**Remarks**	No	

Table of Contents

1. Purpose of the Tutorial 1

2. Dataset Used 1

3. Study Site 1

4. Expected Outcome 1

5. About Linux OS 2

6. Important Commands in Linux OS 2

6.1 Common Useful Commands (at Prompt) 2
6.2 Running multiple commands in one line 6
6.3 Getting information about a command 7
6.4 Short cut keys 7
6.5 General installation and update commands 8
6.6 Web download and installation commands 8
6.7 Remote Access of Linux PC from Linux PC via SSH 8
6.8 File transfer between the PCs 8

7. Installation of Anaconda and its dependencies/packages 9

7.1 Downloading and Installation of Anaconda3 9
7.1.1. Download and install from the web using Terminal window 9
7.1.2. Activate and Test Installation 10
7.1.3. Creation and Activation of Anaconda New Environments 10
7.1.4. Open Jupyter Notebook 11
7.2 Installation of Conda Packages (Required Packages for ISCE) 11

8. Installation of ISCE2 SAR Processor 12

8.1. Web installation directly from conda-forge 12
8.2. Check the version of installed isce2 software 13

9. Installation of MintPy 13

10. Downloading the InSAR Dataset 13

10.1. Imaging Geometry of Sentinel-1 13
10.2. Downloading the Satellite Data from NASA Earth Data Search 15
10.2.1. Login Registration 15
10.2.2. Browsing and Downloading Sentine-1 Datasets 15
10.2.3. Downloading SAR datasets according to Baseline and SBAS 20

11. Pre-Processing for Time-Series Measurement in ISCE2 23

11.1 Prepare the Working Directories 24
11.2 Download Digital Elevation Model (DEM) 24
11.3 Generate Coregistered Stack of SLC 26
11.4 Generate Single Reference Stack in ISCE 32

12. SBAS-InSAR Processing in MintPy Package 33

12.1. Unzipped the data 33
12.2. Open the Jupyter-Notebook 33
12.3. SBAS-InSAR Processing 34

12.3.1 Import the required python modules in Notebook 34
12.3.2 Define the Directories for Processing 35
12.3.3 Unzipping the zipped files 35
12.3.4 Clipping the Area of Interest (AOI) 36
12.3.5 Prepare the Template file for processing 36
12.3.6 Run SBAS-InSAR Time-Series Application 40
12.3.7 Visualization of Outputs/Results 51
12.3.8 Coupling of Ascending and Descending Results 62
12.3.9 Export the Outputs into other formats 63
13. Conclusion 63
Relevant References 64

Tutorial

InSAR Data Processing for Urban City Deformation using Open Source Computing Environment

1. Purpose of the Tutorial

This tutorial aims the analyse of a series of SAR images using a multi-temporal interferometric SAR (InSAR) technique to precisely measure the time-series surface movement (displacement, velocity, and acceleration) of National Capital of India i.e. Delhi city. The analysis is targeted using small baseline subset InSAR (SBAS-InSAR) technique with processing in time-series domain using completely free and open source computing environment. This technique works with pixels of comparatively moderate coherence, called distributed scatterers or slowly-decorrelating filtered phase.

2. Dataset Used

In this tutorial, a stack of 127 Sentinel-1 single look complex images acquired in ascending pass is chosen for processing and analysis. A stack of 488 interferograms will be generated with VV polarization, and 5 × 1 looks along range and azimuth directions respectively. Sentinel-1 images and SRTM DEM 1Sec HGT are downloaded from the ASF website (URL: https://search.asf.alaska.edu/) for a period of around 4 years i.e from January 2021 to March 2025. The selected scenes have a satellite heading angle of -12.5450. The dataset is having maximum perpendicular baseline is 286.83 m, and maximum temporal baseline is 48 day s. The spatial coherence value is ranging between [0.4497, 0.8515] in the study area.

The required dataset and files are provided in a folder Delhi_subsidence.

3. Study Site

Delhi city, the national capital of India, is taken as the study site for this tutorial. The geographical extent of the study site lies between 3158179 m – 3181067 m N latitude and 697848 m – 729788 m E longitude, and the elevation ranges from 132 - 227 m from the mean sea level. Administratively, New delhi is divided into Eleven regions namely North delhi, South Delhi, West Delhi, East Delhi, North-East Delhi, South-East Delhi, North-West Delhi, South-West Delhi, Central Delhi, New Delhi, and Shahdara. There are reports on long-term ground water fluctuations in the Delhi City. This analysis is being done to understand, if there are any land deformation (subsidence and/or upliftment) associated with groundwater variations in the Delhi City.

4. Expected Outcome

At the end of the tutorial, the following outputs are expected-

- Raw phase time-series and line-of-sight (LOS) displacement of Delhi city, India.
- Sequencially corrected phase time-series, and LOS displacement time-series of Delhi City, India.
- SBAS-InSAR measured spatio-temporal LOS displacement, velocity, and acceleration over the study area.

- Visualisation of various intermediate and finally corrected deformation results such as network of interferograms, coherence level, unwrapping errors, geometry files, wrapped and unwrapped interferograms, time-series graphs, displacement, velocity, acceleration, etc.

5. About Linux OS

Ubuntu is arguably one of the most highly regarded and widely used Linux distributions available today. Praised both for its ease of use and reliability, Ubuntu also has a loyal following of Linux users and an active community of developers. The source code that makes up the Ubuntu distribution originates from a highly regarded Linux distribution known as Debian created by Ian Murdoch. If you are new to Linux, or already use Linux and want to try a different Linux distribution it is unlikely you will find a better option than Ubuntu.

If you are running Ubuntu with the GNOME desktop and need to access a command-prompt you will need to open a terminal window. This can be achieved by right-clicking on the desktop background (or any folder/file location in your computer) and selecting the Open Terminal menu option. A terminal window may also be opened within the GNOME desktop using the Ctrl-Alt-T keyboard accelerator. In this tutorial, it is assumed that the Linux OS is already installed in your computer/laptop.

6. Important Commands in Linux OS

When processing SAR datasets, one has to know the following commands for frequent use –

6.1 Common Useful Commands (at Prompt)

- To see the present/current working directory
```
$ pwd
```
- To change the current working directory
```
$ cd /../../../<directory name>
```
- To change the working directory as a shortcut in advance
```
$ HOME_DIR=$/home/iirs/dir1/dir2/  #fixes directory with short name
$ cd  $HOME_DIR      # changes the desired directory as working
directory
```
- Few arguments on *cd* command
```
$ cd ..             # moves one directory up
$ cd ~              # directly come to root directory
$ cd ../../         # goes two step back
$ cd-                # moves into previous directory
```
- To know about all folders and files under present working directory
```
$ ls        # shows all files and folders in current directory
$ ls –1     # shows all files and folders of current directory
$ ls –R      # lists all the files in the sub-directories as well
$ ls –a      # shows the hidden files
$ ls –l      # lists the files and directories with detailed
information like the permissions, size, owner, etc.
$ ls –al     # lists the files and directories with detailed
information like the permissions, size, owner, etc. including hidden
files
```
- Use of cat and touch command
```
$ cat > filename          # creates a new file
$ touch filename            # creates a new file
```

```
$ cat file1 file2>file3   # joins file1 and file2 and store output in a
new file3
$ cat filename | tr a-z A-Z    # converts a file to upper or lower case
use
```

- To copy file from the current directory to a different directory

```
$ cp filename.extension  ~/destination Directory/
$ cp file1 file2   # copy the contents of the first file to the second
file
$ cp –r file1/directory1 ~/directory2  # recursively copy the contents
of the first directory into the second directory
```

- To move /rename file from the current directory to a different directory

```
$ mv filename.extension   ~/destination Directory/
$ mv file1 file2              # renames a file1 to file2
```

- To make a new directory

```
$ mkdir <directoryname>   # creates new directory in current directory
$ mkdir <newdirectory> /home/anotherdirectory/   # creates new
directory in another existing directory
```

- To remove a directory

```
$ rmdir       # removes empty directories in current directory
$ rm –r  ~/<directory name>   # remove a directory recursively
$ rm –rf ~/<directory name>   # remove a directory recursively without
requiring confirmation
```

- To locate a file

```
$ locate –i school    # search a file which contains the word 'school'
anywhere
$ locate –i school*note    # search a file which contains word 'school'
and 'note' anywhere
$ find .-name notes.txt  # search a file notes.txt in current directory
```

- To search a word in a file

```
$ grep blue notepad.txt #searches a word blue in notepad.txt file
```

- To get a report on the system's disk space usage

```
$ df              # report in % and KBs
$ df –m           # report in Megabytes
```

- To check how much space a file or a directory has (File size)

```
$ du filename         # check disk space of a file or directory
$ du ~/filename       # file size in KBs in a given directory
$ du –sh ~/filename   # File or Folder size of a given directory
$ du –ah ~/filename   # shows disk space for all files/directory
```

- To view the head and tail of a text file

```
$ head –n filename.ext       # shows first ten lines
$ head –n 5 filename.ext     # shows first five lines
$ tail –n filename.ext       # shows last ten lines
$ tail –n 5 filename.ext     # shows last five lines
```

- History Command (to review the commands you've entered before)

```
$ history          # To see the command line history in the shell
```

```
$ history | grep <Any Term used in the past>   # To search a specific
term using history and grep
$ history | less    # If you wish to view the history of one page
$ history | tail    # Views just the last ten commands
$ history 25        # Views the last 25 commands
$ history -w ~/history.txt # save bash history manually to a file
$ history –c      # Clear the entire contents of the history file
$ .bash_history   # The history file is stored in a file that you can
modify, as well. Bash shell users find it in their home directory.
```

- There may come a time that you want to remove some or all the commands in your history file. If you want to delete a particular command

```
$ history -d <line number>
$ history –d 16 45     # Deletes all list entries from 16 to 45
$ history –d -8        # Deletes only the last eight commands
```

- Zip and unzip command

```
$ sudo apt-get install zip         # install zip (if not)
$ sudo apt-get install unzip       # install unzip
$ zip options zip_file file1 file2 .. # zip many files into single
$ zip –u zip_file filex       # add a file in existing zip file
$ zip –m zip_file file1 file 2 # deletes original files after zipping
$ zip -e zip_file file1 file2..  # create password protected zipfile
$ unzip options filename.zip    # unzip a file
$ unzip filename.zip –d ~/../   # unzip in a particular directory
$ unzip '*.zip'                 # unzip multiple zip files
$ unzip -P <password> zip_file  # unzip password protected zipfile
```

Options for Zipping:

```
  -u  : Updates the file in the zip archive
  -d  : Deletes the file from the zip archive
  -m  : Deletes original/main files after zipping
  -x  : Exclude files when we are going to create the zip
  -r  : Recursively zip the files inside it and then folders inside it
  -v  : Verbose mode option we will print diagnostic version information
```

- Tar command (Many uses)

```
    $ tar <option(s)> <file(s)>    # general command
    $ tar –xvf  filename.tar    # to untar/extract a tar file (~unzip)
    $ tar –cvf  mytarfile.tar  ~/file1 ~/file2  ~/file3   # creates a
    mytarfile.tar in current directory using three different files
   $ tar –tvf  filename.tar   # view/list the content of a tar file
   $ tar –rvf existingfile.tar  ~/newfile # to append/add a new file in
existing tar file/archive
   $ tar –zcpvf  myarchive.tar.gz  ~/file1 ~/file2  # create and compress a
tar archive file    OR
   $ tar –zcpvf myarchive.tgz ~/file1 ~/file2
   $ tar –zxpvf myarchive.tgz  # extract a compressed tar archive file
```

Options:

```
    --delete           : delete from the archive (not on mag tapes!)
    -r, --append       : append files to the end of an archive
    -t, --list         : list the contents of an archive
    --test-label       : test the archive volume label and exit
    -u, --update       : only append files newer than copy in archive
```

```
-x, --extract, --get     : extract files from an archive
-c, --create       : create a tar file
-C, --directory==DIR     : change to directory DIR
-f, --file=ARCHIVE  : use archive file or device ARCHIVE
-j, --bzip2        : filter the archive through bzip2
-J, --xz           : filter the archive through xz
-p, --preserve-permissions     : extract information about file
permissions
-v, --verbose : verbosely list files processed
-z, --gzip         : filter the archive through gzip
```

- Change read, write and execute permissions of a file
  `$ chmod ugo <file address>    # to give r, w and x permissions`

Where

  - `u=r+w+x, g=r+w+x, o=r+w+x;` Here u, g and o are in octal number system
  - u-user, g=group, o=organization, r=read, w=write, x=execute
  - Octal Notation: 744 means; u=7, g=4, o=4
  - Number r=4 gives read permission, w=2 gives write permission, x=1 gives execute permission and 0 gives no permission
  - Thus 777 gives read, write and execute permission to user, group of users and organization.
    `$ chmod  777 /home/iirs/Project`
    `$ chmod 461 /home/iirs/Project` #gives read permission to user, read and write permission to group, and execute permission to organization

- To check the current jobs and their statuses
  ```
  $ jobs      # jobs and their status in current directory
  ```

- To terminate an unresponsive program manually
  ```
  $ SIGTERM(15)  # requests a program to stop running
  $ SIGKILL(9)   # forces programs to stop immediately. Unsaved progress
  will be lost.
  ```

- To check your internet connectivity status to a server
  ```
  $ ping google.com
  $ ping iirs.gov.in
  ```

- Whoami, Uname, top commands
  ```
  $ whoami       # see root user
  $ uname        # know your machine name, operating system, kernel, and
  so on.
  $ uname –r     # show system information
  $ top          # display a list of running processes and usage of CPU
  by each process
  $ last reboot    # show system reboot history
  $ timedatectl    # query and change the system clock
  $ date           # show the date and local time
  $ uptime         # show how long the system has been running,
  including load average
  ```

- Reissue the last command (Note that the previous command is listed and performed)
  `$ !!     or $!-1`

- To run the penultimate command
  `$ !-2      #similarly other commands !-5, !-10, !-25 and so on`

- Execute the last command that begins with a particular word
```
$ !man   or $ !pwd
```

- Execute the last command that contains a particular word (not necessarily at the line begin)
```
$ !?man?        or $ !?pwd?
```

- To find out where a particular executable resides on the file system
```
$ which ls
$ which history
```

- To save any of your text/statement in a file
```
$ echo hello my name is Raju >> myname.text
```

- To know the name of your host/network
```
$ hostname        # know the hostname
$ hostname –i     # know the network IP
$ ip addr show    # show IP addresses and network interfaces
$ ifconfig        # show IP addresses of all network interfaces
$ netstat –pnltu  # show active ports
$ netstat –nutlp  # show more information about a domain
```

- Adding and Deleting New Users in Linux PC
```
$ useradd  Hari  # add a new user and also set password
$ userdel  Hari  # delete an existing user after giving password
$ passwd           # add a password to a user account
$ id               # show details of the user
$ last             # show the last logins
```

- Running multiple commands in one single command
```
$ Command1; Command2; Command3     # running multiple commands in
parallel
$ Command1 && Command2 && Command3     # if you only want the next
command to run when the first one is successful
```

To see the content of a file
```
$ more filename
```

- To take a snapshot of an active process
```
$ ps
```

- Editing a file
```
$ vim filename
$ nano filename
$ vi filename
```

6.2 Running multiple commands in one line

- **Using semicolon operator (;): Applicable for independent commands**
  The most common practice is using a semicolon (;) between the chain of Linux commands. Using the semicolon (;) operator, we can run multiple in a sequence irrespective of whether the previous command was executed successfully or failed. The basic syntax of using the semicolon (;) operator is:
```
$ command 1; command 2; command 3
```

- **Using && (AND) operator: For sequential execution**
  If we want to run the chain of commands one by one in Linux that is related to the previous command, we can use the AND (&&) operator in this case. The AND operator only allows the user

to run the next command when the previous command is executed successfully. So, we can use the logical AND (&&) operator between the commands in the following way:

```
$ command 1 && command 2 && command 3
```

Example: `$ mkdir my_project cd my_project`
`Output: username:~/my_project$`

- **Using the || (OR) operator: For concatenating the commands**
  The OR (||) the operator allows users to run the next command if the previous command fails and gives an error. But, if the previous command is executed successfully then, the next command will not run. Therefore, the OR (||) operator is used to concatenate multiple commands.

```
$ command 1 || command 2 || command 3
```

Example: `$ mkdir personal || cd personal || pwd`
`mkdir: cannot create directory 'personal': file exists`
`Output: username:~/personal$`

Note: We can also combine Linux commands by combining more than one operator such as AND (&&) and OR (||) operators.

6.3 Getting information about a command

- Use the *man* command specifying the name of the command as an argument.

```
$ man pwd
```

A detailed description of the *pwd* command will be displayed.

- Many commands will also provide additional information when run with the *--help* command-line option:

```
$ wc --help
```

6.4 Short cut keys

```
Clear              # clean the terminal
Tab                # autofill what you are typing
Up Arrow           # see the previous command
Down Arrow         # see the next command
Delete             # Delete character currently beneath the cursor
Backspace          # Delete character to the left of the cursor
Ctrl+b  OR left arrow          # move cursor back one position
Ctrl + f  OR right arrow       # move cursor forward one position
Ctrl+C             # stop and terminate the command (cuurent)
Ctrl+Z             # simply pause the command (currently working)
Ctrl+S             # accidental freezing your terminal
Ctrl+Q             # unfreezing (undo) your freezed terminal
Ctrl+A             # moves the cursor to the beginning of the line
Ctrl+E             # moves the cursor to the end of the line
Ctrl+R             # recall the last command that matches with provided
characters
Ctrl+G             # exit command history without running a command
Env                # show currently defined shell variable
Exit               # log out the current session
```

6.5 General installation and update commands

```
$ pip install <package name>
$ conda install <package name>
$ sudo apt install <package name>
$ sudo apt-get install <package name>
$ sudo apt-get upgrade
$ sudo apt-get update
$ sudo apt install default-jre      #java install
$ sudo dpkg --install
```

6.6 Web download and installation commands

```
$ wget <download link>    # To download some files from internet
$ sudo chmod +x <installer file.ext>   #Make it executable
$ sudo ./<installer_file.sh> # for installation
```

OR download .tar or zip file and untar or unzip file, cd to that untar or unzip file, and run

```
$ sudo ./configure –prefix=<installation directory>; make; make install
$ sudo ./install.sh -p <installation directory>
```

6.7 Remote Access of Linux PC from Linux PC via SSH

- `$ ssh -X <user_name>@<IP Address of remote PC>`
- Type Password for remote PC
- If OpenSSH server is not installed then install it by running: sudo apt install openssh-server openssh-client
- Verify whether ssh service is running: sudo systemctl status ssh
- Configure the firewall: sudo ufw allow ssh
- Check the status of ufw: sudo ufw status
- If ufw is inactive: sudo ufw enable
- Come out /logout from remote PC and access local PC
- To open the file system in remotely accessed PC: run
  `$ nautilus`

6.8 File transfer between the PCs

Linux PC to Linux PC:

- Using SCP command
  `$ scp -r username_hostPC@source_host_IP:~/fileaddress username_RemotePC@RemotePC_IP:~/directory/`

OR

If source PC is alredy logined (doesnt matter physically or remotely) then

```
$ scp -r ~/file username_RemotePC@RemotePC_IP:~/directory/
```

- Using rsync command
  `$ rsync -avz /home/sourceusername/filename destinationusername@destinationIP:/home/destinationusername/`

Windows PC to Linux PC and Vice-Versa:

1) *Using MobaXtern*
   - Download *MobaXterm (Portable edition) (https://mobaxterm.mobatek.net/download-home-edition.html)*, unzip it, and open its application file.

- Start new session, click on SSH, and then give IP, username of Remote Host, click OK.
- A new window will open, give password of Remote Host. Click 'Yes' and then Clicl on 'Cancel'.
- Upload the files/folders in the left side archive.
- These uploaded files/folders can be accessed directly from Linux PC, and anything which is already there (In Linux PC) can also be downloaded in Windows PC.

2) By sharing of Folders

- By sharing the folder of windows. Go to shareable folder in Windows PC > Right click the shareable folder > give access to > specific people..>Enter IP of linux PC > Click on share > provide administrative credentials (username and password) of Windows PC > click done
- Then go to the Linux PC > Files > Other Locations > connect to server and enter smb://IPofWindowsPC/ShareableFolderName> Click connect. This will connect the Shareable folder.
- New popup window will come, select registered_user, and enter username and password of linux pc and click connect.
- Now you are free to put any file/folder under this shareable folder from any PC i.e. connected Linux and Windows PC. And copy any file/folder in any PC from this folder.
- **Note:** from windows2linux 'smb' is required in Windows PC and from linux2windows 'samba' is required in linux PC.

7. Installation of Anaconda and its dependencies/packages

7.1 Downloading and Installation of Anaconda3

Anaconda is a package manager used in scientific computing and data science. It's designed to provide scientific libraries and dependencies in the Python programming language. Anaconda is commonly used for machine learning and artificial intelligence applications.

7.1.1. Download and install from the web using Terminal window

- Open the terminal window (Ctrl-Alt-T)
- Download the Latest Version of Anaconda: Check the developer's download page to view the newest version.
- Right click on newest version and *Copy link address* and use it to download the correct version.
- Change the current working directory where you want to download the latest version of Anaconda3.

```
$ cd /home/<user>/…/Desired working directory
```

- Use wget to download the installer using your command terminal:

```
$ wget https://repo.anaconda.com/archive/Anaconda3-2024.10-1-Linux-x86_64.sh
```

- Run the Installation Script of downloaded Anaconda3: The Anaconda installer is a **bash script**. To run the installation script, use the command-

```
$ bash ~/Desired working directory/Anaconda3-2024.10-1-Linux-x86_64.sh
```

  - A license agreement will appear. Use the **Enter** key to review the agreement.
  - At the bottom, type **yes** to agree to the terms.

The installer will prompt you to accept the default location, or install to a different location. Use the default path unless you have a specific need to change it. (You may cancel the installation here if needed). The installation will finish. After successful installation, the following will appear:

installation finished.

Do you wish the installer to prepend the Anaconda3 install location to PATH in your /home/user/.bashrc? [yes|no]

7.1.2. Activate and Test Installation

- Once installation finished, activate the installation by entering:
  `$ source ~/.bashrc`
- Use the **conda** command to test the installation:
  `$ conda info`

The system should display a list of data similar to:

```
     active environment : base
    active env location : /home/iirs/miniconda3
            shell level : 1
       user config file : /home/iirs/.condarc
 populated config files : /home/iirs/miniconda3/.condarc
          conda version : 25.1.1
    conda-build version : not installed
         python version : 3.12.9.final.0
                 solver : libmamba (default)
       virtual packages : __archspec=1=icelake
                          __conda=25.1.1=0
                          __glibc=2.39=0
                          __linux=5.15.167.4=0
                          __unix=0=0
      base environment : /home/iirs/miniconda3  (writable)
     conda av data dir : /home/iirs/miniconda3/etc/conda
  conda av metadata url : None
          channel URLs : https://repo.anaconda.com/pkgs/main/linux-64
                         https://repo.anaconda.com/pkgs/main/noarch
                         https://repo.anaconda.com/pkgs/r/linux-64
                         https://repo.anaconda.com/pkgs/r/noarch
         package cache : /home/iirs/miniconda3/pkgs
                         /home/iirs/.conda/pkgs
      envs directories : /home/iirs/miniconda3/envs
                         /home/iirs/.conda/envs
              platform : linux-64
            user-agent : conda/25.1.1 requests/2.32.3 CPython/3.12.9 Linux/5.15
.167.4-microsoft-standard-WSL2 ubuntu/24.04.1 glibc/2.39 solver/libmamba conda-l
ibmamba-solver/25.1.1 libmambapy/2.0.5 aau/0.5.0 c/. s/. e/.
               UID:GID : 1000:1000
            netrc file : None
          offline mode : False
```

7.1.3. Creation and Activation of Anaconda New Environments

- Create a Python 3 environment named ***isce2*** by entering the following:
  `$ conda create --name isce2 python=3`

OR

```
$ conda create –n isce2 python=3
```
OR
```
$ conda create -n <envname> -c conda-forge --file requirements.txt
```

- Once created, activate this environment:
  ```
  $ conda activate isce2
  ```

The command prompt will change. This indicates that you are now in a shell environment using Python 3. You can now work in this environment for the entire course.

7.1.4. Open Jupyter Notebook

- Change the working directory as –
  ```
  $ cd ~/homedirectory/anaconda3
  ```
- To open Jupyter-Notebook, run the following command-
  ```
  $ jupyter-notebook
  ```

Note:

1) Change jupyter-notebook directory in the terminal window itself (if required). The directory of terminal window is the directory of jupyter notebook.
2) If jupyter notebook not installed in new environment, then install its by

```
$ conda install -c conda-forge notebook
```
or
```
$ conda install -c conda-forge jupyterlab
```

7.2 Installation of Conda Packages (Required Packages for ISCE)

We recommend setting us a new virtual environment (see section 3.1.3) specifically meant for building and installing ISCE. If you intend to use the installed version with other packages, just install the other packages into the same environment at the very beginning.

Preparing requirements.txt file and Installation of packages

Prepare a simple .txt file with the following list of conda packages required for ISCE

- cython
- gdal
- git
- h5py
- libgdal
- pytest
- numpy
- fftw
- scipy
- basemap
- scons
- opencv

- Keep the requirements.txt file wherever you want to create new environment (in this case ~/anaconda3/envs/). And after creation of new environment and installation of above packages.

```
$ cd ~/anaconda3/envs/
$ conda create -n <envname> -c conda-forge --file requirements.txt
```

- Now you need to activate this newly created environment.
- Activate the installed virtual environment

```
$ conda activate nameofmyisceenv
```

Notes:

i) Installation of the above packages can also be installed one by one using the command

```
$ conda install <package name>    OR
$ conda apt-get install <package name>   OR
$ pip install <package name>
```

ii) To clean the cache of PC, use the following command –

```
$ conda clean --index-cache --lock --tarballs --packages –source-cache
```

iii) To see the list of installed conda and/or pip packages in your PC –

```
$ conda list
$ pip list installed
```

8. Installation of ISCE2 SAR Processor

ISCE stands for **I**nSAR (Interferometric Synthetic Aperture Radar) **S**cientific **C**omputing **E**nvironment. Its initial development was funded by NASA's Earth Science Technology Office (ESTO) under the Advanced Information Systems Technology (AIST) 2008 and is currently being funded under the NASA-ISRO SAR (NISAR) project. ISCE software is an open source under the terms of the Apache License. Its export classification is 'EAR99 NLR', which entails some restrictions and responsibilities. Please read the accompanying LICENSE.txt and LICENSE-2.0 files.

ISCE is a framework designed for the purpose of processing InSAR data. The framework aspects of it have been designed as a general software development framework. It may have additional utility in a general sense for building other types of software packages. In its InSAR aspect ISCE supports data from many space-borne satellites and one air-borne platform. Development team continue to increase the number of sensors supported by ISCE. Currently, ISCE supports processing of data acquired by following platforms: ALOS, ALOS-2, COSMO_SKYMED, ENVISAT, ERS, KOMPSAT-5, RADARSAT-1, RADARSAT-2, RISAT-1, Sentinel-1, TERRASAR-X, TANDEM-X, UAVSAR and SAOCOM-1A.

One can install ISCE2 software using any one of the following process-

8.1. Web installation directly from conda-forge

Before installing ISCE2 from the web, you should ensure that newly created environment (say isce2) is activated. Then start installing the isce2 using the following command in terminal (before running this command, make sure your PC is connected with the internet).

```
$ conda install -c conda-forge isce2
                OR
$ conda install -c conda-forge isce2=version
                OR
$ conda install -c conda-forge isce2=version python=version
```

- For more detail you may visit the following web page https://anaconda.org/conda-forge/isce2
- Install isce2 with a specific version of python in the active virtual environment
  ```
  $ conda install -c conda-forge isce2 python=version
  ```
- Check the installation
  ```
  $ stackSentinel.py –h
  ```
- If getting any error, then export the PATHs
  ```
  $export
  PATH=$PATH:~/anaconda3/envs/isce2/share/isce2/topsStack/:~/anaconda3/en
  vs/isce2/share/isce2/prepStackToStaMPS/bin/:~/anaconda3/envs/isce2/lib/
  python3.9/site-packages/isce/applications/:
  ~/anaconda3/envs/isce2/lib/python3.9/site-packages/isce/
  ```
- Run again for checking the installation
  ```
  $ stackSentinel.py –h
  ```

8.2. Check the version of installed isce2 software

```
$ python
>>> import isce
>>> isce.version.release_version
```

9. Installation of MintPy

The Miami INsar Time-series software in PYthon (MintPy) is an open-source package for InSAR time series analysis. It reads the stack of interferograms (coregistered and unwrapped) in ISCE, ARIA, FRInGE, HyP3, GMTSAR, SNAP, GAMMA or ROI_PAC format, and produces 3D (2D in space and 1D in time) ground surface displacement in line-of-sight (LOS) direction. It includes a routine time series analysis (smallbaselineApp.py) and some independent toolbox.

Before installing MintPy from the web, you should ensure that you have created a new conda environment for MintPy installation (say mint), and activate the same newly created environment. Then start installing the MintPy using the following command in terminal (before running this command, make sure your PC is connected with the internet).

MintPy is available on the *conda-forge* channel. The latest released version can be installed via *conda* as:

```
$ conda install -c conda-forge mintpy
```

If you want to install a specific version of MintPy, run the following-

```
$ conda install -c conda-forge mintpy=version
```

For checking the installation, run the following command-

```
$ smallbaselineApp.py -h
```

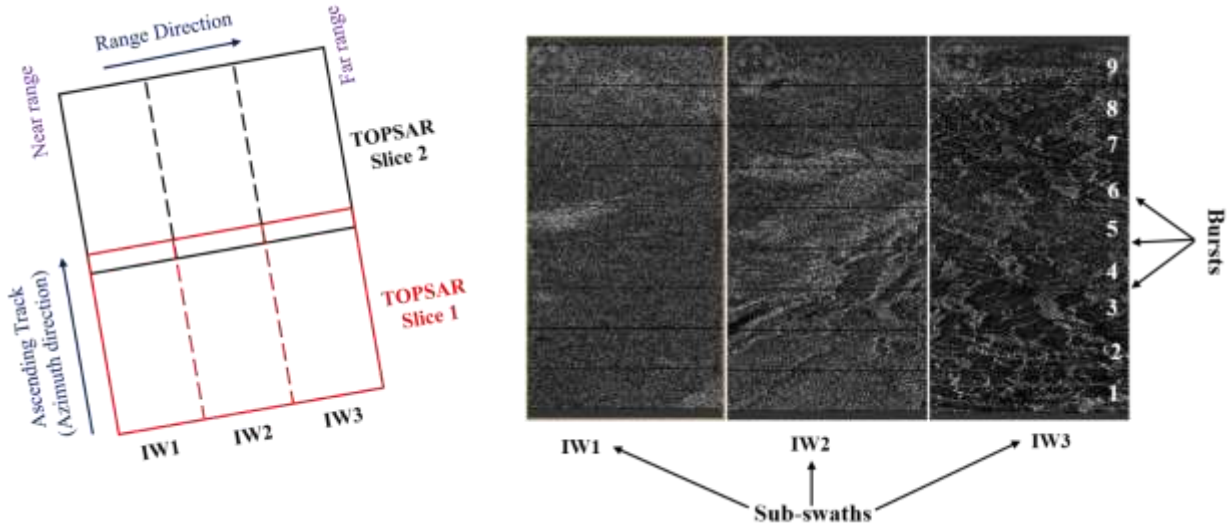
Create a New Python Kernel in virtual environment for MintPy processing using jupyter notebook, as –

```
$ conda install –c anaconda/miniconda ipykernel
$ python –m ipykernel install --user --name=<name of environment>
```

10. Downloading the InSAR Dataset

10.1. Imaging Geometry of Sentinel-1

Sentinel-1 is a constellation of two satellites i.e. Sentinel-1A and Sentinel-1B operating in C- band (5.405 GHz). The constellation reduces the repeat frequency of the satellites from 12 to 6 days over a particular area of interest and thus making the data more useful for interferometric and polarimetric applications. Interferometric Wide swath (IW) is one of the acquisition modes of Sentinel-1 acquiring data over the land surface. In this mode, three sub-swaths (IW1, IW2, and IW3) images per polarization channel are imaged using Terrain Observation with Progressive Scans SAR (TOPSAR) technique with a spatial resolution of 5 x 20 m. Thus for single polarization there are total of three images i.e. IW1, IW2 and IW3 images; for dual polarization channel total six images (three for each polarization). TOPSAR mode replaces the conventional ScanSAR mode, achieving the same coverage and resolution as ScanSAR, but with a nearly uniform SNR (Signal-to-Noise Ratio) and DTAR (Distributed Target Ambiguity Ratio).



As the name of TOPS indicates, the radar sensor performs a scan of the surface by electronically steering the antenna beam from a backward-pointing along-track direction to a forward-pointing along-track direction for a fixed range swath (also called a subswath). After a successful scan at that range extent, the antenna beam is electronically rolled back to its initial position, the range swath is electronically directed outward to a new area to increase coverage, and the next scan is made from backward to forward along track at this new range swath. After a third scan at a third range swath, the entire process is repeated over and over to create a continuous image as the satellite flies along. The timing of the scans is such that there is a small geographic overlap between successive scans at a given range to ensure continuous coverage. Each scan at a given range swath is known as a "burst". When inspecting one of the downloaded SLC products you will note that data are provided in 3 individual subswaths (IW1, IW2, IW3) each with a set of bursts. All together they form a Sentinel-1 frame, typically ~250 x 250 km in size. The bursts are approximately 20 km in length and overlap by 2 km. Due to the TOPS acquisition mode, the overlap region in successive bursts is seen from two different directions (forward looking and backward looking).

The IW products are delivered in the form of slices to make the data more manageable in terms of storage and computational effort. Each slice (image scene) is a stand-alone product and can be processed independently. Each slice covers approximately 250 x 150 km in range and azimuth direction (single look). The nomenclature of each sub-swath is based on the orbit type, i.e. ascending or descending. The sub-swath close to the near range is IW1, and the sub-swath in the far range is IW3. Each sub-swath image consists of nine bursts in the azimuth direction, where each burst has been processed as a separate SLC image. The individually focused complex bursts are arranged in azimuth-time order with the black-fill demarcation

(gap) between them. Above Figure represents the imaging geometry of an ascending pass Sentinel-1 IW product.

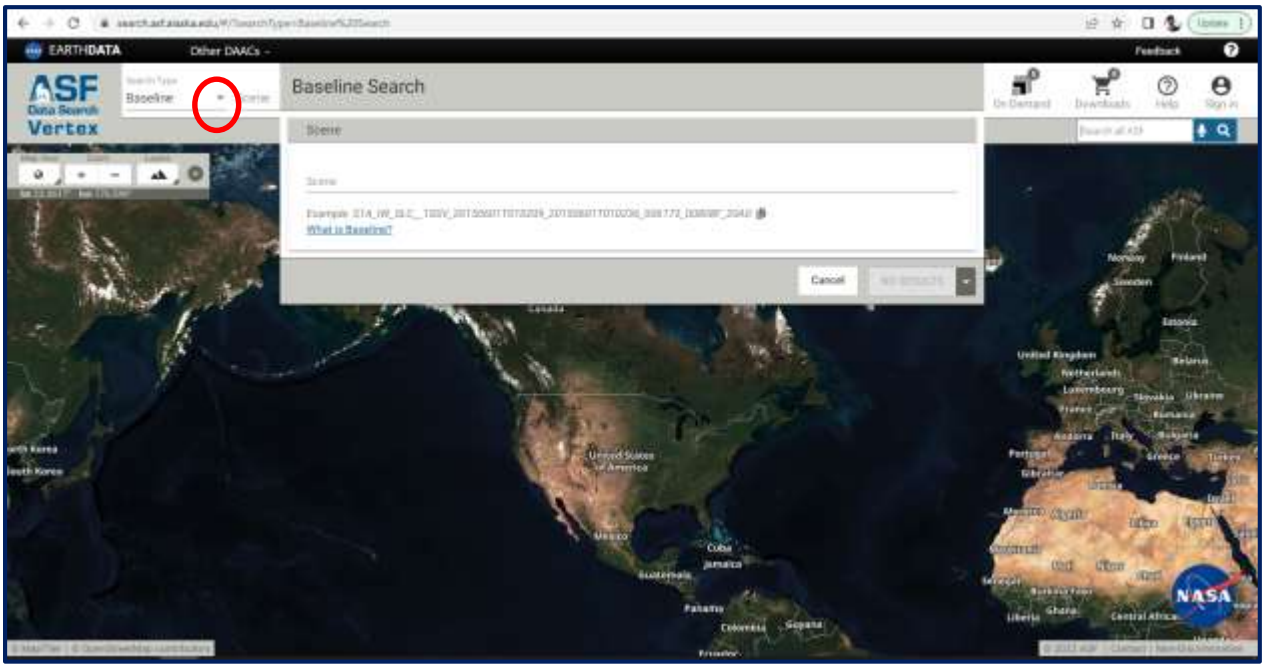
10.2. Downloading the Satellite Data from NASA Earth Data Search

10.2.1. Login Registration

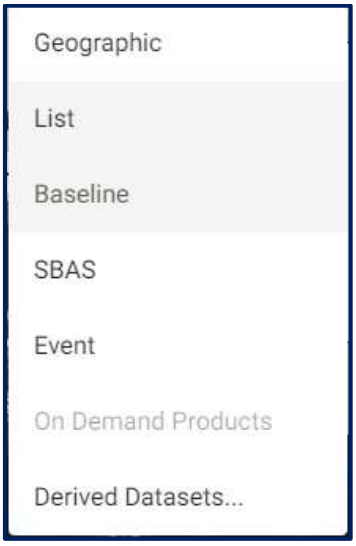
Create your login account by registering on the https://urs.earthdata.nasa.gov/ and sign in with your login credentials.

10.2.2. Browsing and Downloading Sentine-1 Datasets

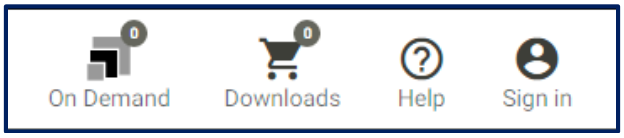
- Go to Alaska Satellite Facility of NASA Earth data search site https://asf.alaska.edu/. A number of datasets are available under DAAC (Distributed Active Archive Center).
- Click either on *VERTEX* or *Interferometry* icon given in the bottom of the webpage. You will get the following interface.



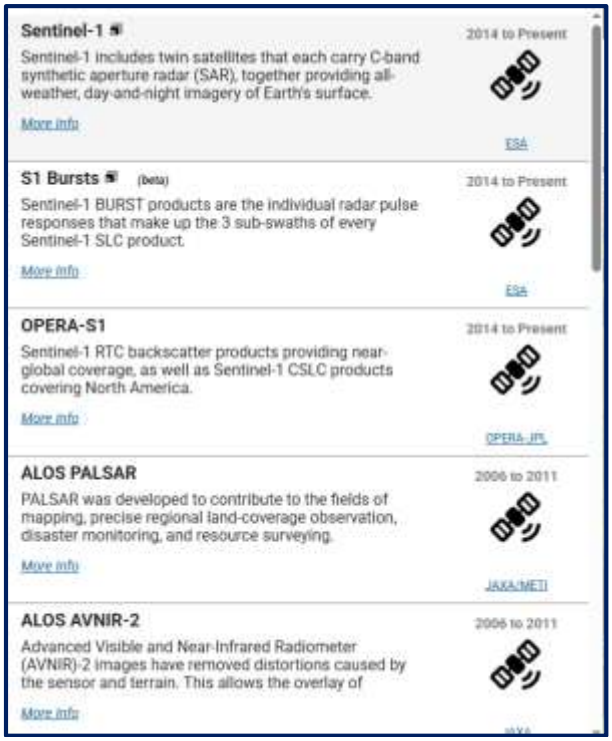
- Click on the drop down icon under Search Type. Where you will get the following list of search types.



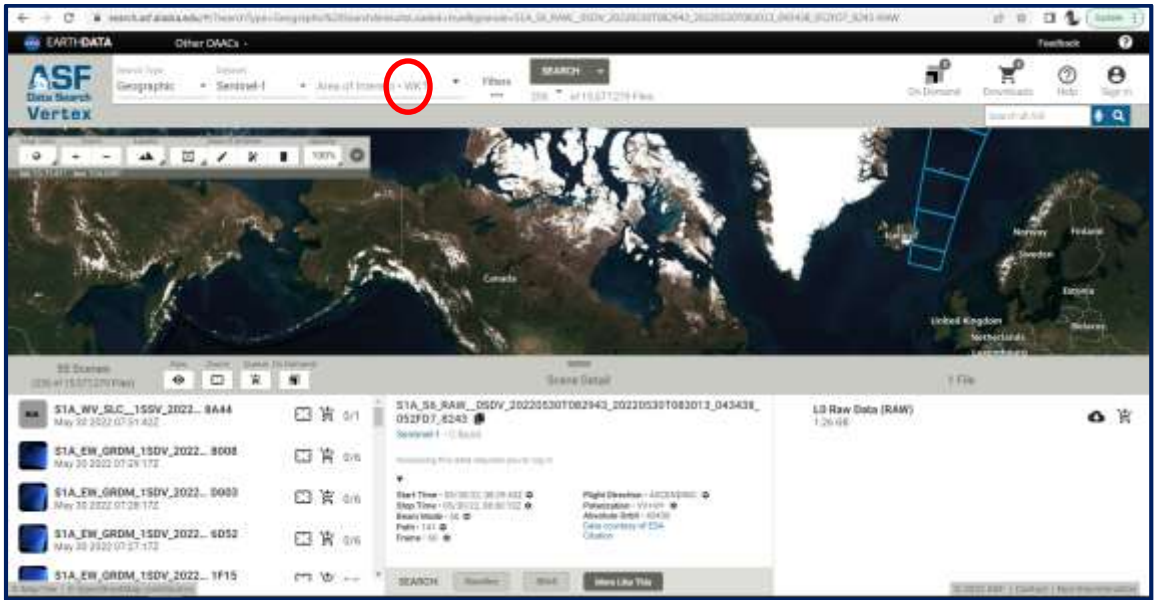
- Now first of all you have to sign in on the webpage by clicking on *Sign in* icon and entering the Login credentials.



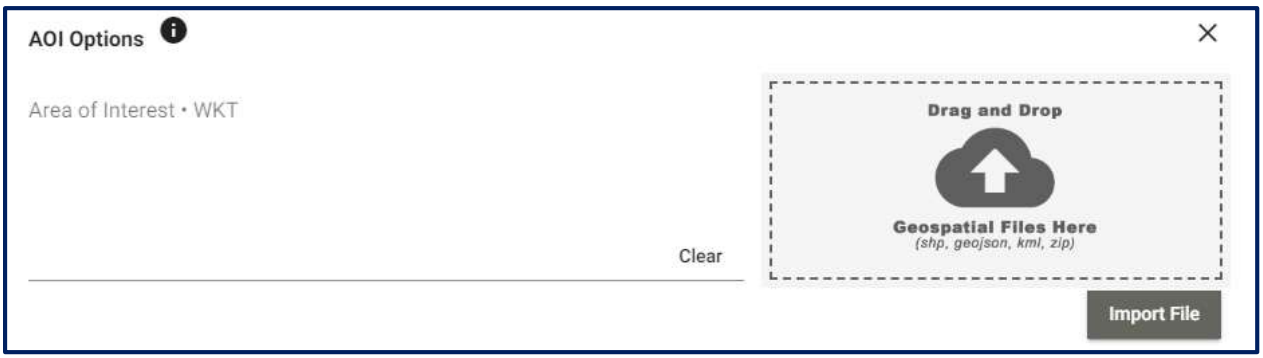
- Select the search type as per your requirement. Let first select *'Geographic'* under search Type. Now click on drop down list of next icon i.e. Dataset. You will get a dropdown list including below options.



- Since we are interested in time-series dataset from space borne SAR sensor for this course, and thus Select *Sentinel-1* (for large AOI) or *S1 Bursts* (for smaller AOI) from this drop down list.



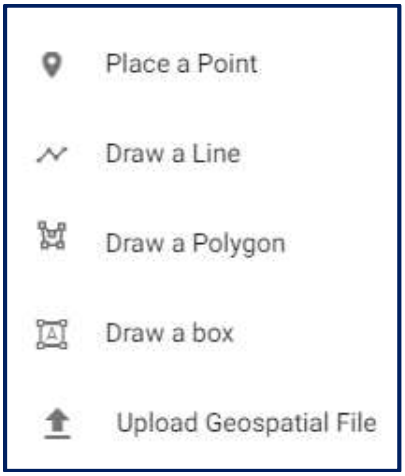
- Click on the more options under *Area of Interest-WKT*, and drag and drop a Geospatial layer/file i.e. shapefile (.shp) or geojson file (.geojson) or .kml or .zip file of your study area.



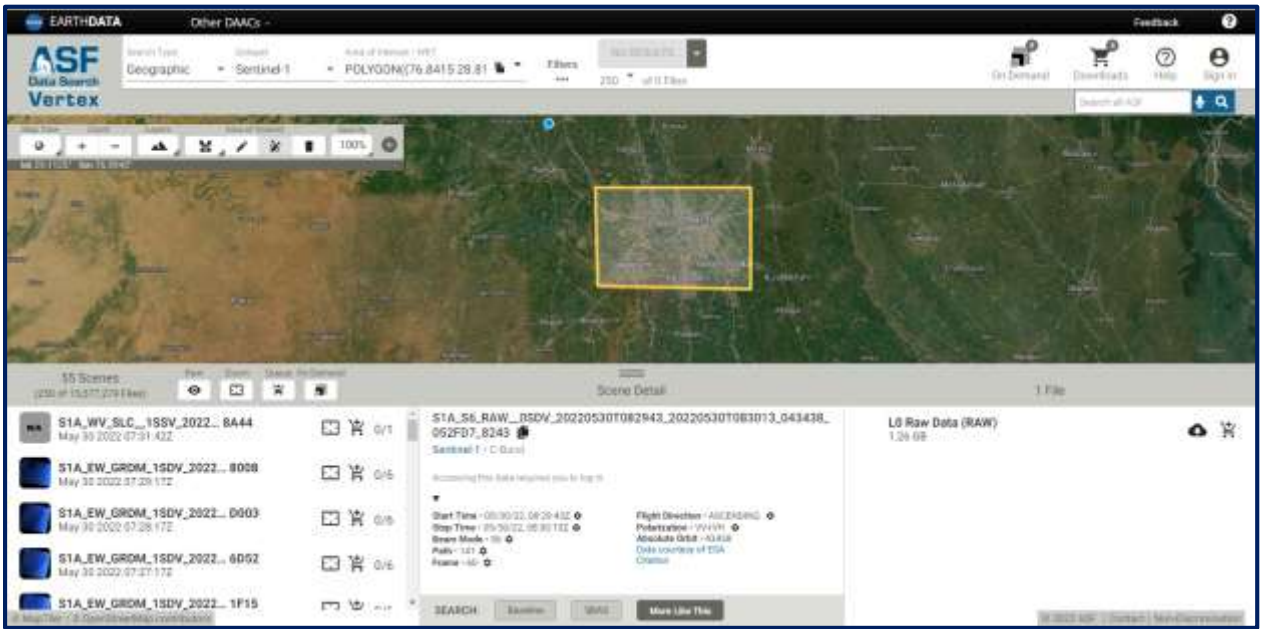
- If you don't have readymade geospatial layer, then you can create a polygon around your area of interest using the following tool.



- You will have the following options to select.



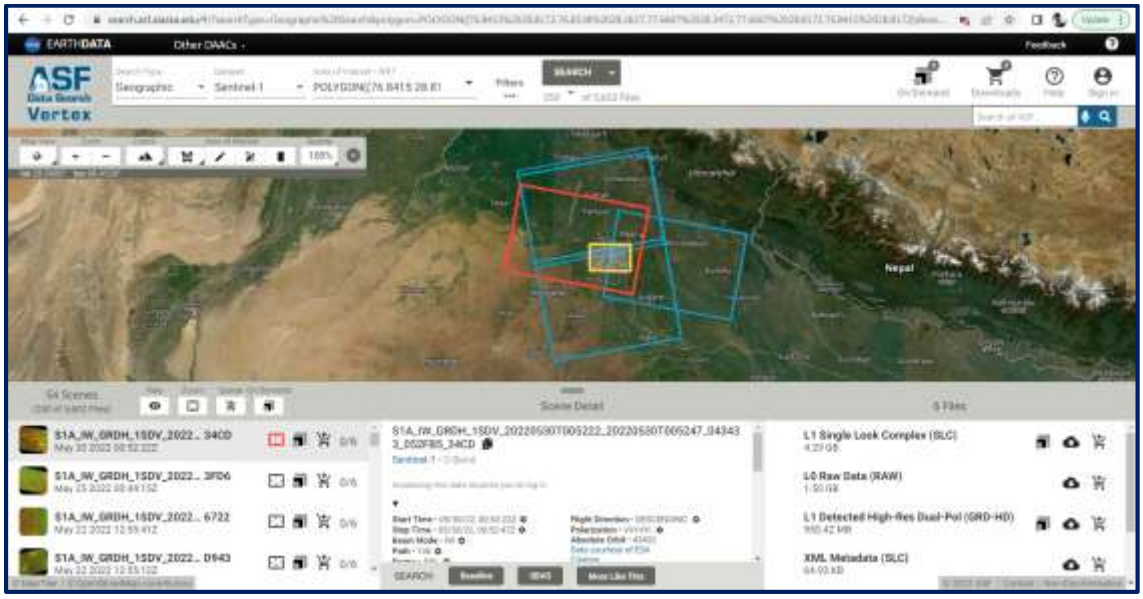
- Scroll the satellite image in background and zoom in your area of interest, and Select *'Draw a Polygon'* option.



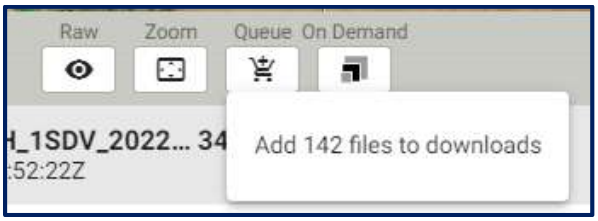
- Click on 'Filter' tool and then click on Search.



- After clicking on *Search*, the corresponding image frames will be highlighted on the interface as well as in the left section of bottom panel. The Scene detail and corresponding number of files are also shown in the middle and right sections of the bottom panel.



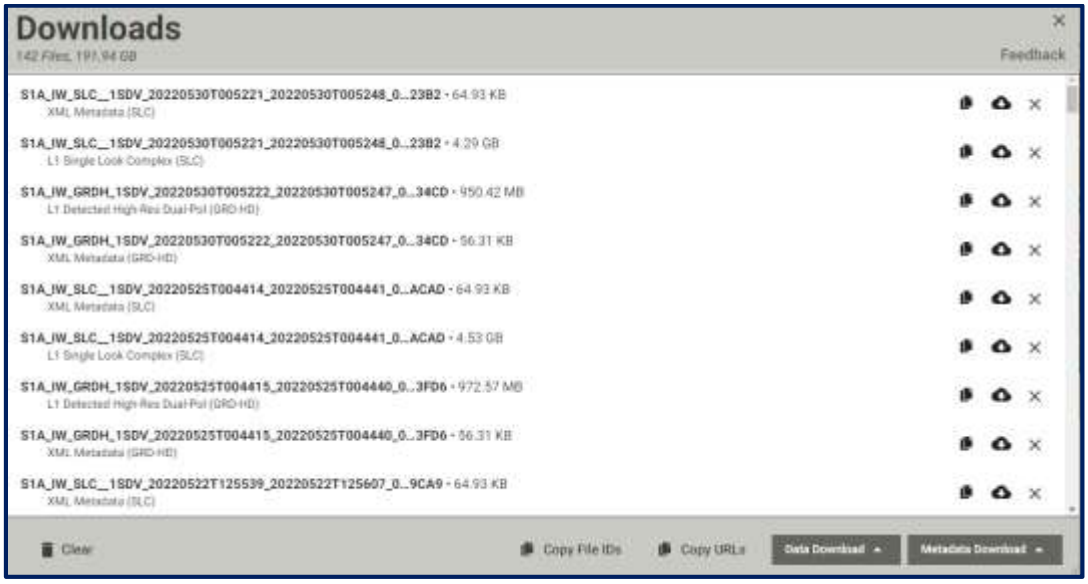
- Click on *Queue* and again click on *Add No. of Files to downloads.*



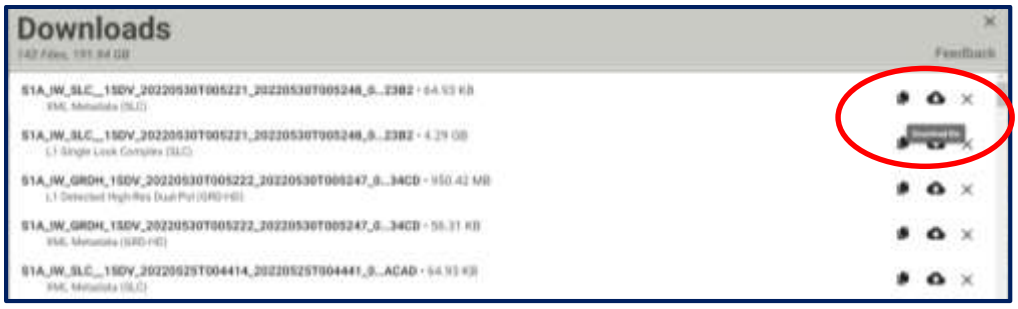
- Now you will be able to see the above no. of files are added to the *Downloads (i.e. Cart).*



- Now click on *Downloads* and a list of all selected scenes will be seen.



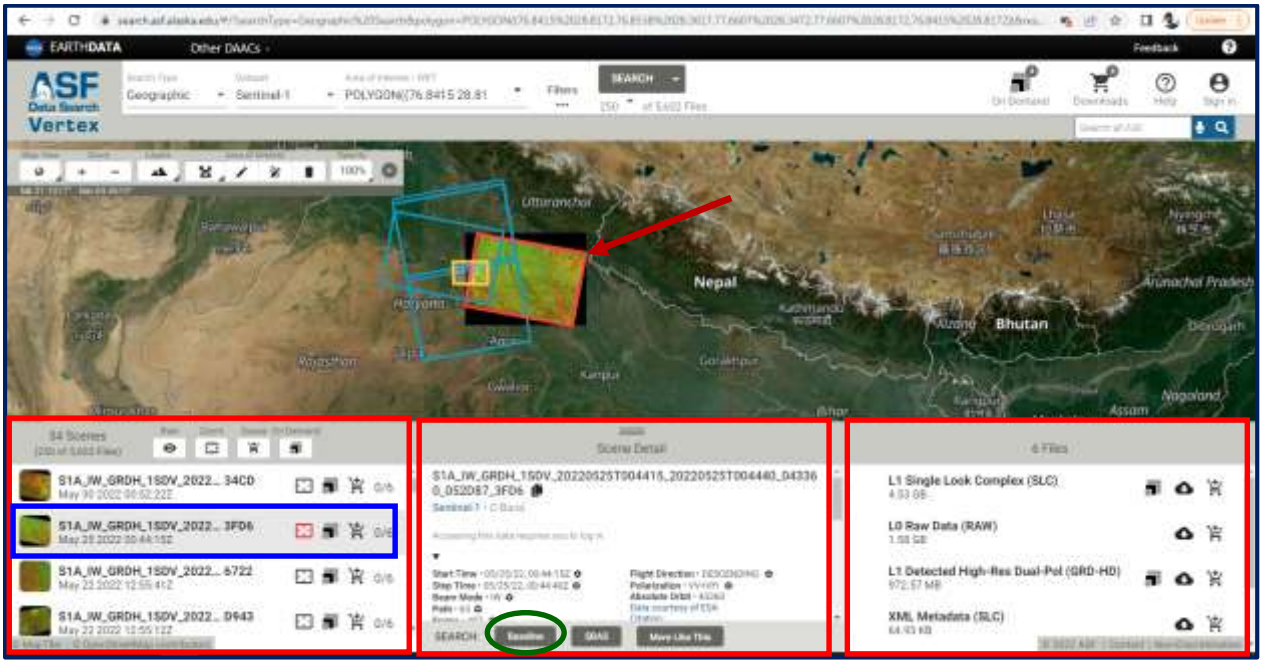
- Now by clicking on *Download file* (down arrow), you can download the respective scene. Make sure you are Logged in on the web page.



- You can click on multiple scenes simultaneously for download. Thus your download for a Sentinel-1 SAR data is completed.

10.2.3. Downloading SAR datasets according to Baseline and SBAS

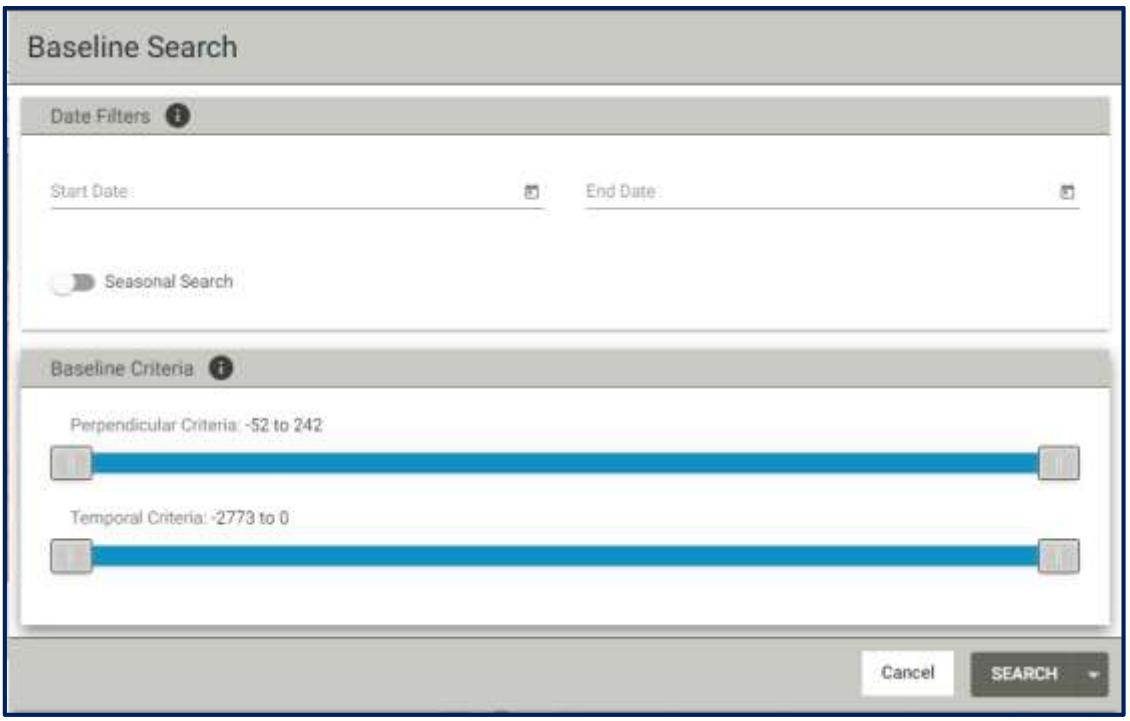
- If you are interested in interferometry datasets that means time-series datasets. For this, click on Clear button on the bottom left corner of the *Downloads* window which is currently opened and then close the widow. Now you can see no image scene is added in the *Downloads* cart.
- Now select an image scene (perfectly overlapping with your area of interest) on the left section of bottom panel (shown by blue rectangle) of computer screen, and this image will act as also reference image. The moment you select a scene of your choice, you will observe three things- i) Corresponding image scene will be highlighted on the web interface (shown by red arrow), ii) Scene's detail will be shown in the middle section of the bottom panel (shown by red rectangle), and iii) Number of files corresponding to selected scene will be highlighted in the right section of bottom panel (shown by red rectangle).



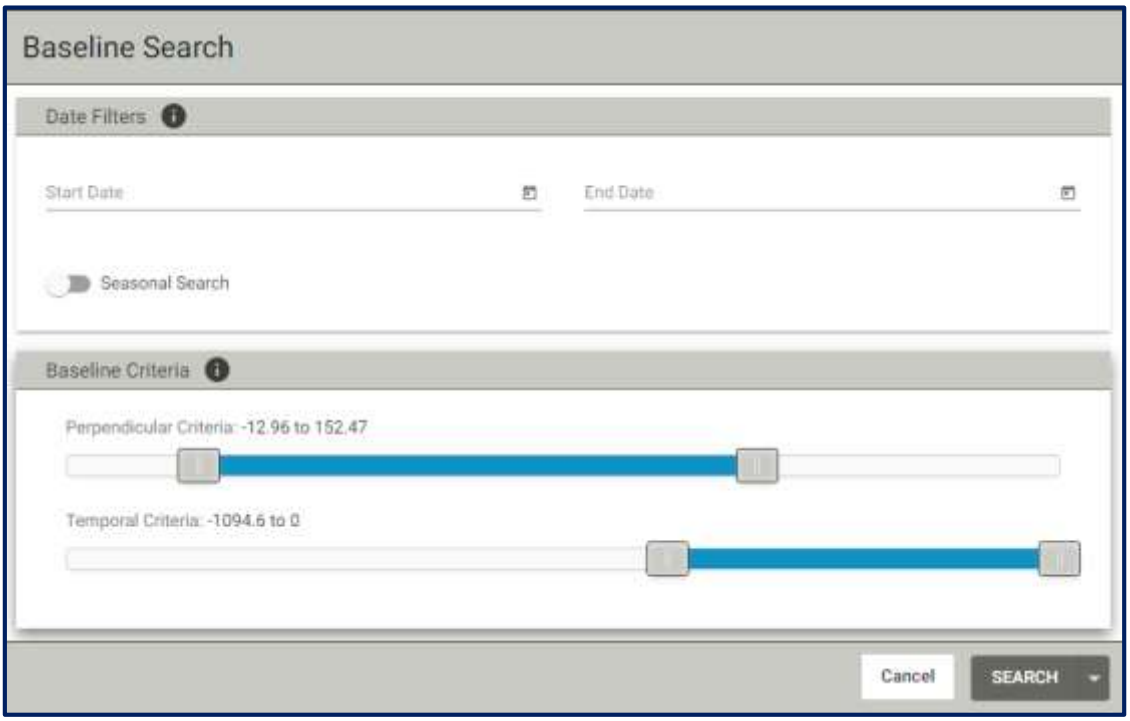
- Now select *Baseline* icon (shown by green ellipse) in the middle section of bottom panel. Here you will see again three things- i) Scene frames representing different baselines corresponding to previously selected scene i.e. reference image (highlight on the interface), ii) All baseline scenes on the left section of bottom panel, and iii) Baselines graph (Perpendicular baseline versus Temporal Baseline) on the right section of bottom panel.



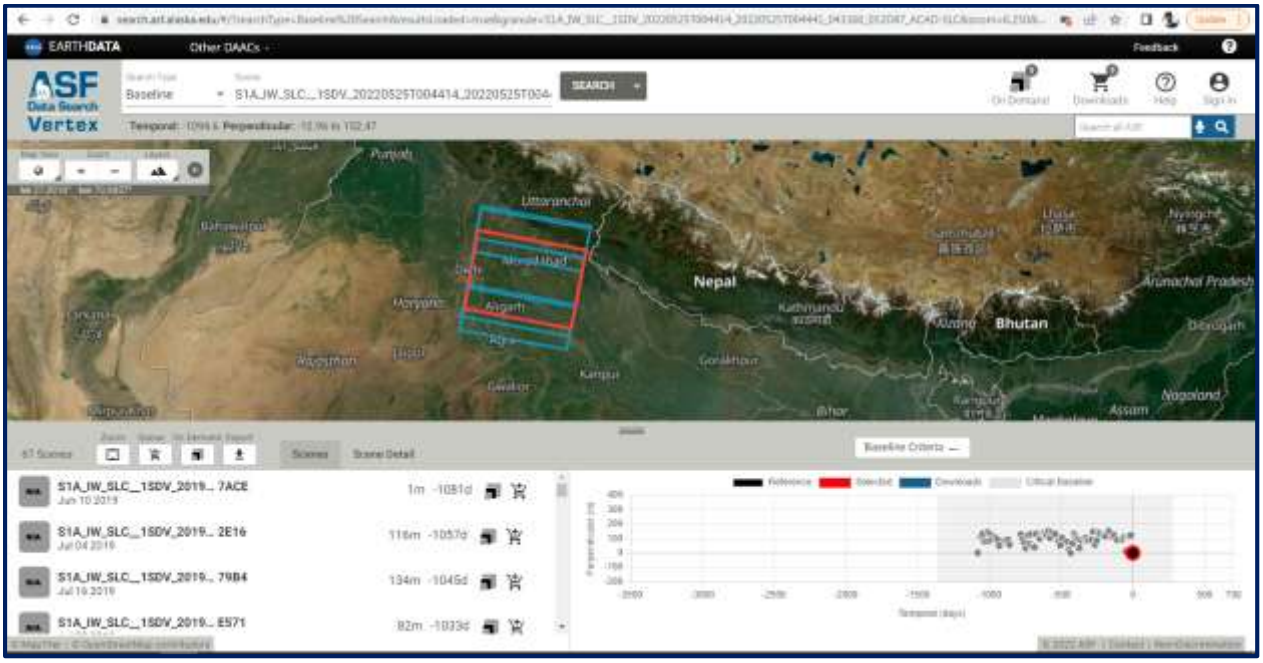
- The baselines can be modified as per your requirement with respect to perpendicular and temporal baselines. For this click on *Baseline Criteria*.



- Now you can change perpendicular and temporal baselines as per your choice. Let's select perpendicular baseline between -12.96 to +152.47, and temporal baseline between -1094.6 to 0. Here zero means reference image and -1094.6 represents number of days before reference image.

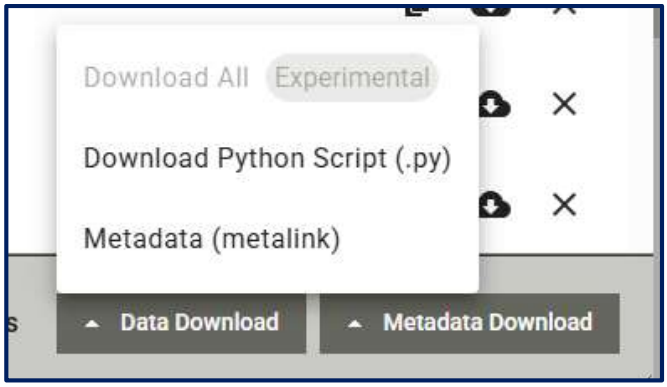


- Click on *SEARCH*.



- Now here you can see the selected images are as per your choice. Now for downloading these selected images, click on *Queue* > click on *Add … files to_download*. All images will be added in the *Downloads* and start downloading them one-by-one or multiple images or using python API file.
- For downloading datasets for SBAS analysis, you need to click on *SBAS* in place of *Baseline* (on the middle section of bottom panel) and rest of the process is same.

If you want to download using python API, then first add all the images/processed images under download section, and then first download the 'Download Python Script' from the bottom of download section under Data Download tab.



- Once the python script is downloaded, then run the following command in Linux terminal, and type the user name and password of ASF NASA Earth Data.

```
$ python <file name>
```

11. Pre-Processing for Time-Series Measurement in ISCE2

In PSI processing, we need a coregistered stack and subsequently a stack of single reference/master interferograms. Moreover, in this exercise, the script provides the support for Sentinel-1 TOPS image stack processing. This task can be done in ISCE2 using topsStack processing workflow. To use the sentinel stack processor, make sure to add the path of your contrib/stack/topsStack folder to your $PATH environment variable. Currently supported workflows include a coregistered stack of SLC, interferograms, offsets, and coherence.

11.1 Prepare the Working Directories

In order to process Sentinel-1 TOPS Level-1 SLC data for time-series deformation measurements, first of all you need to create a working directory to start processing which may further consists of sub-directories. First create a main directory i.e. My_Project. Under this main directory create three sub-directories – i) SLCs to store all downloaded the Sentinel-1 SLC images, ii) DEM to store the DEM (downloaded) of the study area, and iii) Outputs to store the processing results of the workflow(s)-

```
$ cd /home/username
$ mkdir My_Project
$ cd My_Project
$ mkdir SLCs
$ mkdir DEM
$ mkdir Outputs
```

Download and Put all SLC image files (.zip files) in the SLCs folder. One another important input is required for stackSentinel.py workflows which is precise orbit files. These precise orbit can be downloaded from https://scihub.copernicus.eu/gnss/ manually and can be putted in another sub-directory under Outputs directory OR if you are executing stackSentinel.py then these precise orbit files will be automatically (Missing orbits downloaded on the fly) downloaded by this script provided your computer has internet connectivity. For this you have check URLs in fetchOrbit.py and dloadOrbits.py files under topsStack folder. Another folder within the Output sub-directory may be created to store Sentinel-1 auxiliary files which is used for correcting the Elevation Antenna Pattern (For this Orbit sub-directory may also be used).

11.2 Download Digital Elevation Model (DEM)

Download of DEM (need to use wgs84 version) using the ISCE DEM download script. This you can do by two ways –

i) Using dem.py script

```
$ cd ~/My_project/DEM
$ dem.py –h                    # see the workflow and parameters
```

Usage

dem.py [-h] [-a ACTION] [-c] [-m META] [-s SOURCE] [-f] [-v FILLINGVALUE] [-b BBOX

[BBOX ...]] [-p PAIRS [PAIRS ...]][-k] [-r] [-l] [-d DIR] [-o OUTPUT] [-n UNAME] [-w PASSWORD]

[-t TYPE] [-x] [-u URL]

Optional arguments:

Argument	Description
-h OR --help	show this help message and exit
-a ACTION OR --action ACTION	Possible actions: stitch or download (default: stitch)

-c OR --correct	Apply correction EGM96 -> WGS84 (default: False). The output metadata is in xml format only
-m META OR --meta META	What type of metadata file is created? Possible values: xml or rsc (default: xml)
-s SOURCE OR --source SOURCE	Dem SRTM source. Possible values 1 or 3 (default: 1)
-f OR --filling	Flag to instruct to fill missing DEMs with null values (default null value -32768. Use -v or --filling_value option to change it)
-v FILLINGVALUE OR --filling_value FILLINGVALUE	Value used to fill missing Dems (default: -32768)
-b BBOX [BBOX ...] OR --bbox BBOX [BBOX ...]	Defines the spatial region in the format south north west east. The values should be integers from (-90,90) for latitudes and (0,360) or (-180,180) for longitudes.
-p PAIRS [PAIRS ...] OR --pairs PAIRS [PAIRS ...]	Set of latitude and longitude pairs for which –action = download is performed. The values should be integers from (-90,90) for latitudes and (0,360) or (-180,180) for longitudes
-k OR --keep	If the option is present then the single files used for stitching are kept. If -l or --local is specified than the flag is automatically set (default: False)
-r OR --report	If the option is present then failed and succeeded downloads are printed (default: False)
-l OR --local	If the option is present then use the files that are in the location specified by --dir. If not present --dir indicates the directory where the files are downloaded (default: False)
-d DIR OR --dir DIR	If used in conjunction with --local it specifies the location where the DEMs are located otherwise it specifies the directory where the DEMs are downloaded and the stitched DEM is generated (default: ./)
-o OUTPUT OR --output OUTPUT	Name of the output file to be created in --dir. If not provided the system generates one based on the bbox extremes
-n UNAME OR --uname UNAME	User name if using a server that requires authentication
-w PASSWORD OR --password PASSWORD	Password if using a server that requires authentication
-t TYPE OR --type TYPE	Use version 3 or version 2 SRTM, or nasadem
-x OR --noextras	Use this flag if the filenames do not have extra part
-u URL OR --url URL	If --type=version2 then this is part of the url where the DEM files are located. The actual location must be the one specified by --url plus /srtm/version2_1/SRTM(1,3).If --type=version3 then it represents the full path url

Usage examples:

1: Stitch (-a stitch) 1 arcsec dems (-s 1) in the bounding region 31 33 82 85 using the url (-u) and the log in credentials provided (-n,-w). Create a rsc metadata file (-m) and report the download results (-r).

```
$ dem.py -a stitch -b 31 33 82 85 -s 1 -m rsc -r -n your_username -w your_password -u https://aria-alt-dav.jpl.nasa.gov/repository/products/SRTM1_v3/
```

2: Download (-a download) the 3 arcsec (-s 3) whose lat/lon are 30 77 and 31 78 (-p)

```
$ dem.py -a download -p 30 31 77 78 -s 3
```

3: Stitch the requested files and apply EGM96 -> WGS84 correction (-c)

```
$ dem.py -a stitch -b 30 31 77 78 -r -s 1 –c
```

4: Download from bounding boxes (-b)

```
$ dem.py -a download -b 30 31 77 78 -r  -s 1
```

5: Stitch the files in the local directory (-l) in the bounding region provided keeping the zip files after stitching (-k)

```
$ dem.py -a stitch -b 30 31 77 78 -k  -r -l  -s 1
```

Note: The following URLs can be used for downloading the required DEM-

- https://aria-alt-dav.jpl.nasa.gov/repository/products/SRTM1_v3/
- https://urs.earthdata.nasa.gov/
- https://earthexplorer.usgs.gov/
- https://e4ftl01.cr.usgs.gov/MEASURES/SRTMGL1.003/2000.02.11/

Now stitch (download) the DEM and apply correction for processing purpose-

```
$ dem.py -a stitch -b 30 31 77 78 -r -s 1 -c
$ rm demLat*.dem demLat*.dem.xml demLat*.dem.vrt
```

ii) Using tiling of the DEMs

Download DEMs tiles (see the extend of your study region and find range of latitude and longitude e.g. N= 30-31 and E=77-78) one-by-one using wget as:

```
$ wget --user=your username --password=your password
http://e4ftl01.cr.usgs.gov/MEASURES/SRTMGL1.003/2000.02.11/N30E077.SRTMGL1.hgt.zip
$ wget --user=your username --password=your password
http://e4ftl01.cr.usgs.gov/MEASURES/SRTMGL1.003/2000.02.11/N30E078.SRTMGL1.hgt.zip
$ wget --user=your username --password=your password
http://e4ftl01.cr.usgs.gov/MEASURES/SRTMGL1.003/2000.02.11/N31E077.SRTMGL1.hgt.zip
$ wget --user=your username --password=your password
http://e4ftl01.cr.usgs.gov/MEASURES/SRTMGL1.003/2000.02.11/N31E078.SRTMGL1.hgt.zip
```

After downloading all the tiles within the study region, you can merge all downloaded DEM Tiles by running dem.py file as

```
$ dem.py -a stitch -l -b 30 31 77 78 -r -s 1 -c -k --user=your username --
password=your password
$ rm demLat*.dem demLat*.dem.xml demLat*.dem.vrt
```

Note: If you have stitched the DEMs tiles into one file and used once as its original position. And next time if this DEM folder is shifted to some other location and using it in running run_files, then before using it, one has to change the location directory of .xml file of DEM as per new location in your PC, otherwise it will show error message.

11.3 Generate Coregistered Stack of SLC

`stackSentinel.py` generates all configuration and run files required to be executed on a stack of Sentinel-1 TOPS data. When stackSentinel.py is executed for a given workflow (-W option) a `configs` and `run_files` folder is generated. No processing is performed at this stage. Within the run_files folder different `run_#_description` files are contained which are to be executed as shell scripts in the run number order. Each of these run scripts call specific configure files contained in the "configs" folder which call ISCE in a modular fashion. The configure and run files will change depending on the selected workflow. To make run_# files executable, change the file permission accordingly (e.g., chmod +x run_01_unpack_slc).

```
$ stackSentinel.py -H    #see workflow examples,
$ stackSentinel.py -h    #get an overview of all the configurable parameters
```

Required parameters of stackSentinel.py include:

```
$ stackSentinel.py -h
```
usage:

```
$ stackSentinel.py [-h] [-H] -s SLC_DIRNAME -o ORBIT_DIRNAME -a AUX_DIRNAME
[-w  WORK_DIR] -d DEM [-m REFERENCE_DATE] [-c NUM_CONNECTIONS] [-n
SWATH_NUM] [-b BBOX] [-t TEXT_CMD] [-x EXCLUDE_DATES] [-i INCLUDE_DATES] [--
start_date STARTDATE] [--stop_date STOPDATE] [-z AZIMUTHLOOKS] [-r
RANGELOOKS] [-f FILTSTRENGTH] [--snr_misreg_threshold SNRTHRESHOLD] [-p
POLARIZATION] [-C {geometry,NESD}] [-O NUM_OVERLAP_CONNECTIONS] [-e
ESDCOHERENCETHRESHOLD] [-W {slc, correlation, interferogram, offset}] [-V
{True, False}] [-useGPU] [--num_proc NUMPROCESS][--num_proc4topo
NUMPROCESS4TOPO] [-u {icu, snaphu}] [-rmFilter]
```

Preparing the directory structure and config files for stack processing of Sentinel data Optional arguments:

Argument	Description
-h, --help	show this help message and exit
-H, --hh	Display detailed help information.
-s SLC_DIRNAME, --slc_directory SLC_DIRNAME	Directory with all Sentinel SLCs
-o ORBIT_DIRNAME, --orbit_directory ORBIT_DIRNAME	Directory with all orbits
-a AUX_DIRNAME, --aux_directory AUX_DIRNAME	Directory with all aux files
-w WORK_DIR, --working_directory WORK_DIR	Working directory (default: ./)
-d DEM, --dem DEM	Directory with the DEM referenced to wgs84
-m REFERENCE_DATE, --reference_date REFERENCE_DATE	Directory with reference acquisition
-c NUM_CONNECTIONS, --num_connections NUM_CONNECTIONS	number of interferograms between each date and subsequent dates (default: 1)
-n SWATH_NUM, --swath_num SWATH_NUM	A list of swaths to be processed. -- Default : '1 2 3'
-b BBOX, --bbox BBOX	Lat/Lon Bounding SNWE. -- Example : '30 32 77.6 80.2' -- Default : common overlap between stack
-t TEXT_CMD, --text_cmd TEXT_CMD	text command to be added to the beginning of each line of the run files (default: ''). Example : 'source ~/.bash_profile;'

-x EXCLUDE_DATES, --exclude_dates EXCLUDE_DATES	List of the dates to be excluded for processing. -- Example : '20220803,20211231' (default: None)
-i INCLUDE_DATES, --include_dates INCLUDE_DATES	List of the dates to be included for processing. -- Example : '20220803,20211231' (default: None)
--start_date STARTDATE	Start date for stack processing. Acquisitions before start date are ignored. format should be YYYY-MM-DD e.g., 2020-10-15
--stop_date STOPDATE	Stop date for stack processing. Acquisitions after stop date are ignored. format should be YYYY-MM-DD e.g., 2022-08-03
-z AZIMUTHLOOKS, --azimuth_looks AZIMUTHLOOKS	Number of looks in azimuth for interferogram multi-looking (default: 3)
-r RANGELOOKS, --range_looks RANGELOOKS	Number of looks in range for interferogram multi-looking (default: 9)
-f FILTSTRENGTH, --filter_strength FILTSTRENGTH	Filter strength for interferogram filtering (default: 0.5)
--snr_misreg_threshold SNRTHRESHOLD	SNR threshold for estimating range misregistration using cross correlation (default: 10)
-p POLARIZATION, --polarization POLARIZATION	SAR data polarization (default: vv)
-C {geometry,NESD}, --coregistration {geometry,NESD}	Coregistration options (default: NESD)
-O NUM_OVERLAP_CONNECTIONS, --num_overlap_connections NUM_OVERLAP_CONNECTIONS	number of overlap interferograms between each date and subsequent dates used for NESD computation (for azimuth offsets misregistration) (default: 3).
-e ESDCOHERENCETHRESHOLD, --esd_coherence_threshold ESDCOHERENCETHRESHOLD	Coherence threshold for estimating azimuth misregistration using enhanced spectral diversity (default: 0.85)
-W {slc,correlation,interferogram,offset}, --workflow {slc,correlation,interferogram,offset}	The InSAR processing workflow (default: interferogram)
-V {True, False}, --virtual_merge {True,False}	Use virtual files for the merged SLCs and geometry files. Default: True for correlation / interferogram workflow False for slc / offset workflow
-useGPU, --useGPU	Allow App to use GPU when available
--num_proc NUMPROCESS, --num_process NUMPROCESS	number of tasks running in parallel in each run file (default: 1)
--num_proc4topo NUMPROCESS4TOPO, --num_process4topo NUMPROCESS4TOPO	number of parallel processes (for topo only) (default: 1)
-u {icu,snaphu}, --unw_method {icu,snaphu}	Unwrapping method (default: snaphu)
-rmFilter, --rmFilter	Make an extra unwrap file in which filtering effect is removed

Note that *stackSentinel.py* does not process any data. It only prepares a lot of *input files* for processing and a lot of *run files*. Then you need to execute all those generated run files in order i.e., running run_03 needs results from run_02, etc. To know what is really going on, after running *stackSentinel.py*, look at each *run file* generated by *stackSentinel.py*. Each run file actually has several commands that are independent from each other and can be executed in parallel. The *config files* for each *run file* include the processing options to execute a specific command/function.

Examples: stackSentinel.py can be run for different workflows including:

- a stack of interferogram
- a stack of correlation files

- a stack of offsets
- a coregistered stack of SLC

Workflow can be chosen with -W option.

Example 1: Interferogram workflow with 2 nearest neighbor connections (default coregistration is NESD):
It generates the run and configure files needed to generate a stack of interferograms. In this example, a stack of interferograms is requested for which up to 2 nearest neighbor connections are included.

```
$ stackSentinel.py -s ~/SLC/ -d ~/DEM/demLat_N30_N31_Lon_E77_E80.dem.wgs84 -b
'30 31 77 78' -a ~/AuxDir/ -o ~/Orbits -c 2
```

Example 2: Interferogram workflow with all possible interferograms and coregistration with only geometry:
In this example, all possible interferograms are being generated and in which the coregistration approach is set to use geometry and not the default NESD.

```
$ stackSentinel.py -s ~/SLC/ -d ~/DEM/demLat_N30_N31_Lon_E77_E80.dem.wgs84 -b
'30 31 77 78' -a ~/AuxDir/ -o ~/Orbits -C geometry -c all
```

Example 3: Correlation workflow with all possible correlation pairs and coregistration with geometry:

It generates the run and configure files needed to generate a stack of coherence. In this example, a correlation stack is requested considering all possible coherence pairs and where the coregistration approach is done using geometry only. This workflow is basically similar to the previous one, but only difference is that the interferograms are not unwrapped.

```
$ stackSentinel.py -s ~/SLC/ -d ~/DEM/demLat_N30_N31_Lon_E77_E80.dem.wgs84 -b
'30 31 77 78' -a ~/AuxDir/ -o ~/Orbits -C geometry -c all -W correlation
```

Example 4: slc workflow that produces a coregistered stack of SLCs

Run and configure files needed to generate a coregistered stack of SLCs. In this example, a pre-defined bounding box is specified. Note, if the bounding box is not provided it is set by default to the common SLC area among all SLCs. We recommend that you always set the processing bounding box. Here is an example command to create configuration files for a stack of SLCs:

```
$ stackSentinel.py -s ~/SLC/ -d ~/DEM/demLat_N30_N31_Lon_E77_E80.dem.wgs84 -b
'30 31 77 78' -a ~/AuxDir/ -o ~/Orbits -C NESD  -W slc
```

Note: For all workflows, coregistration can be done using only *geometry* or with geometry plus refined azimuth offsets through NESD approach. Existing workflows: slc, interferogram, correlation, offset.

Now in continuation of this exercise, lets create a coregistered stack of SLC using stackSentinel-py with reference slc, sub-swath, coherence threshold and number of processors for parallel processing-

```
$ cd ~/Outputs
```

```
$ stackSentinel.py -s ~/SLC/ -d ~/DEM/demLat_N30_N31_Lon_E77_E80.dem.wgs84 -b
'30 31 77 78' -a ~/AuxDir/ -o ~/Orbits -C geometry -W slc –m ‘20220803’ –n
‘1’ –e 0.65 --num_proc 4
```

Probable Error

1) From topsStack.Stack import config, run, sentinelSLC
ModuleNotFoundError: No module name ‘topsStack’

This is change introduce in the Stack Processor. This error starts when you are working with the latest version of ISCE2. And fixed it with the following correction

From Stack import config, run, sentinelSLC

2) You can also setup the $PYTHONPATH as well in addition to the $PATH, as

```
export ISCE_STACK={full_path_of_your_contrib/stack}
export PYTHONPATH=${PYTHONPATH}:${ISCE_STACK}
export PATH=${PATH}:${ISCE_STACK}/topsStack
```

Example:

```
export ISCE_STACK=/home/iirs/miniconda3/envs/isce2/lib/python3.9/site-
packages/isce/components/isceobj/Stack
export PYTHONPATH=${PYTHONPATH:${ISCE_STACK}
export PATH=${PATH}:${ISCE_STACK}/topsStack
```

3) IndexError: list index out of range

This is error due to orbit files. The Copernicus Scihub client has discontinued service in favor of the new Copernicus Data Space Ecosystem. The new service no longer allows anonymous public downloads (using the gnssuser), which means you must register for either a Dataspace account (to use the CDSE data) or a NASA Earthdata account (to use the orbits provided by ASF). Visit the following link for more detail: https://github.com/scottstanie/sentineleof

- First create a .netrc file under orbits folder or in home directory as: touch .netrc
- Open the .netrc file by clicking the file or using any editor say vi or nano.
- Add the credentials of Copernicus Data Space Account or NASA Earthdata. As

```
machine dataspace.copernicus.eu
  login MYUSERNAME
  password MYPASSWORD
machine urs.earthdata.nasa.gov
  login MYUSERNAME
  password MYPASSWORD
```

Give the permission (atleatt 600) to the .netrc file

```
$ chmod 777 ~/.netrc
```

Then install sentineleof:

```
$ conda install –c conda-forge sentineleof
```

Download the Orbit files by running the command

```
$ eof --search-path /path/to/safe_files/ --save-dir ./orbits/
```

Note:

1) This command will automatically download the Orbit files for Each SLC image and will store in a orbits folder under Output folder, but make sure (https://scihub.copernicus.eu/gnss/#/home) https://dataspace.copernicus.eu/ is assigned to dloadOrbit.py and fetchOrbit.py files; Every time you have to download new orbit files otherwise you will get errors in *sh run_files*; So better not to assign orbit files in orbits and allow to download automatically)
2) The calibration auxiliary (AUX_CAL) file is used for antenna pattern correction to compensate the range phase offset of SAFE products with IPF version 002.36 (mainly for images acquired before March 2015). If all your SAFE products are from another IPF version, then no AUX files are needed.
3) Make sure, all the paths of *stackSentinel.py* are correct otherwise it will show errors while running *run_files*. If such errors are coming, you need to re-run *stackSentinel.py* again after deleting previous outputs.
4) One can directly download the orbit files from: Precise orbits https://s1qc.asf.alaska.edu/aux_poeorb/ and Restituted orbits https://s1qc.asf.alaska.edu/aux_resorb/ OR http://step.esa.int/auxdata/orbits/Sentinel-1/

After running the *stackSentinel.py* command above, the *configs* and *run_files* folders will be created. User needs to execute each *run file* in order. The order is specified by the index number of the run file name. After this, run these files one-by-one (in chronological order) as

```
$ cd run_files
$ sh run_01_unpack_topo_reference
$ run_02_unpack_secondary_slc
and so on
$ cd ..
```

Note: You can give a single command to run all these files in a sequential order (see section 4.3 of exercise 1) as (Not recommended for understanding purpose)

```
$ sh first run_file && sh second run_file && sh third run_file && sh fourth
run_file && sh fifth run_file $$ sh sixth run_file && sh seventh run_file &&
sh eigth run_file and so on.
```

Probable Error 1:

IndexError: too many indices for array: array is 1-dimensional, but 2 were indexed

***Possible solution**:* This sometimes happens when you are working with old orbit files. Therefore, don't use older orbit files and download fresh ones.

Probable Error 2:

module 'numpy' has no attribute 'int'.
np.int was a deprecated alias for the builtin int. To avoid this error in existing code, use int by itself. Doing this will not modify any behavior and is safe. When replacing np.int, you may wish to use e.g. np.int64 or np.int32 to specify the precision. If you wish to review your current use, check the release note link for additional information. The aliases was originally deprecated in NumPy 1.20; for more details and guidance see the original release note at: https://numpy.org/devdocs/release/1.20.0-notes.html#deprecations

Running: topo ['--reference', '/scratch/09401/xlv/EifelSenDT139_SP/reference', '--dem', '/scratch/09401/xlv/EifelSenDT139_SP/DEM/demLat_N46_N54_Lon_E004_E012.dem.wgs84', '--geom_referenceDir', '/scratch/09401/xlv/EifelSenDT139_SP/geom_reference', '--numProcess', '8'] running in parallel with 8 processes
Traceback (most recent call last):
File "/work2/09401/xlv/stampede2/software/rsmas_insar/tools/miniconda3/share/isce2/topsStack/SentinelWrapper.py", line 251, in main(args.start,args.end)
File "/work2/09401/xlv/stampede2/software/rsmas_insar/tools/miniconda3/share/isce2/topsStack/SentinelWrapper.py", line 242, in main cfgParser.runCmd()
File "/work2/09401/xlv/stampede2/software/rsmas_insar/tools/miniconda3/share/isce2/topsStack/SentinelWrapper.py", line 54, in runCmd func_modules.main(self.funcParams[section])
File "/work2/09401/xlv/stampede2/software/rsmas_insar/tools/miniconda3/share/isce2/topsStack/topo.py", line 112, in main bbox = [np.min(boxes[:,0]), np.max(boxes[:,1]), np.min(boxes[:,2]), np.max(boxes[:,3])]
IndexError: too many indices for array: array is 1-dimensional, but 2 were indexed

Possible Solution: There is a np.int in the following file

Lib/python/site-packages/isce/components/isceobj/Sensor/TOPS/Sentinel1.py. Replace this np.int as np.int64 or np.int32 depending upon the precision.

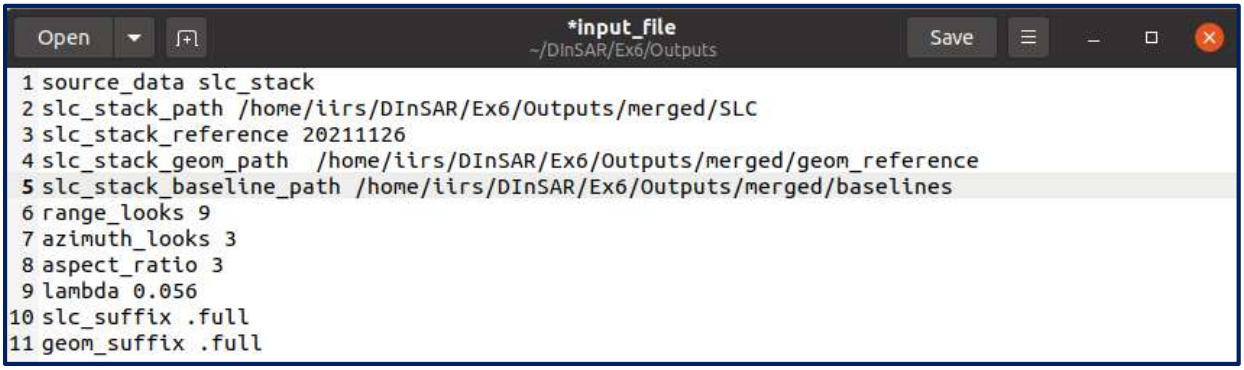
11.4 Generate Single Reference Stack in ISCE

Generate a file called ***'input_file'*** *(a text file)* at the location where the INSAR_reference_date folder needs to be generated (normally under My_Project/Outputs/). The following parameters should be contained in the *'input_file'*. Required parameters are indicated with (R), optional parameters with (O). The reference date can be chosen based on minimizing perpendicular and temporal baselines. The baselines will be automatically re-inverted to the chosen reference.

Parameter	**Value**	**Description** ***Required (R), *Optional (O)**
source_data	slc stack	Do not change (R)
slc_stack_path	PATH_SLC	Path to the SLC stack (R)
slc_stack_reference	yyyymmdd	Reference date as to be used in StaMPS processing. Does not need to be the same as the reference used for generating the coregistered stack. (R)
slc_stack_geom_path	PATH_geom	Path to the geometry directory (R)
slc_stack_baseline_path	PATH_baseline	Path to the baselines directory (R)
range_looks	9	Number of range looks multi-looked products (R)
azimuth_looks	3	Number of azimuth look multi-looked products (O). azimuth_looks overrules aspect_ratio.
aspect_ratio	3	Aspect ratio multi-looked products (R)
lambda	0.056	Wavelength in meter units (R)

slc_suffix	.full	Suffix string for SLC's (O)
geom_suffix	.full	Suffix string for geom files (O)

The overview of *'input_file' will look like this*



To generate the single reference setup type:

```
$ make_single_reference_stack_isce input_file
```

This will create a INSAR_Reference_date folder in which many files and folders will be there. After this, you are ready to perform PSI, SBAS and hybrid of PSI & SBAS processing.

12. SBAS-InSAR Processing in MintPy Package

Before processing the SBAS-InSAR in Mintpy, ensure the following things-

- Anaconda or miniconda is installed in your PC.
- MintPy package and required python packages are installed in the separate conda environment in your computer.
- SAR processor (ISCE or any other as mentioned above) is installed in your PC. If the pre-processed data is downloaded from ASF website using ON DEMAND facility (Hyp3 SAR Processor), then no SAR processor is required to install in your PC.
- The time-series SAR data (SLC), DEM, and other required data (if any) is downloaded.

12.1. Unzipped the data

If the SAR data (unwrapped interferograms) is downloaded from the ASF On demand facility (see section 6.2.3), then unzip all them

```
$ cd /<directory in which downloaded zip files are available>
$ unzip '*.zip'
```

Delete all the zip files as now you have corresponding unzipped files (time-series) of unwrapped interferograms.

12.2. Open the Jupyter-Notebook

Before opening the jupyter-notebook, you should ensure that – i) anaconda is installed in your PC, ii) MintPy is installed in new environment (say mint), iii) Jupyter-notebook is installed in newly created environment, iv) python kernel is created inside the newly created environment for running the notebook.

- Activate the conda environment created for MintPy

```
$ conda activate mint
```

- Change the working directory as –

```
$ cd <desired working directory for InSAR processing>
```

- To open Jupyter-Notebook, run the following command-

```
$ jupyter-notebook
```

If jupyter notebook is not installed in new environment, then install it by

```
$ conda install -c conda-forge notebook
```

or

```
$ pip install notebook
```

- The jupyter-notebook will be opened in the default browser

12.3. SBAS-InSAR Processing

- Open the *InSAR_Processing.ipynb* file provided to you (browse working directory) in the jupyter-notebook by double clicking on the file. OR download the same file from URL: https://github.com/insarlab/MintPy-tutorial/blob/main/workflows/smallbaselineApp_hyp3.ipynb

- Select the python kernel as Kernel > Change Kernel > Newly Created Kernel

12.3.1 Import the required python modules in Notebook

```
%matplotlib inline
import os
import numpy as np
import h5py
from pathlib import Path
from typing import List, Union
from osgeo import gdal
from osgeo import osr
import numpy as np
from numpy import arange
import mintpy
from mintpy import workflow, view, tsview, plot_network, plot_transection,
plot_coherence_matrix
from mintpy.objects import ifgramStack
from mintpy.utils import readfile, writefile, plot as pp, utils as ut
from mintpy.cli import view, tsview, plot_network, plot_transection
from mintpy.unwrap_error_phase_closure import
calc_num_triplet_with_nonzero_integer_ambiguity
from mintpy.view import prep_slice, plot_slice
# from mintpy.objects.insar_vs_gps import plot_insar_vs_gps_scatter
from cartopy import crs as ccrs
from zipfile import ZipFile
from zipfile import BadZipFile
import json
import csv
import matplotlib as mpl
```

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import matplotlib.lines as mlines
import matplotlib.colors as mcolors
from matplotlib.dates import DateFormatter
from matplotlib.ticker import ScalarFormatter
from matplotlib.colors import ListedColormap
from matplotlib.ticker import FuncFormatter
from matplotlib.lines import Line2D
import math
import urllib
import requests
import datetime
import pandas as pd
import geopandas as gpd
from geopandas import GeoSeries
from geopandas import GeoDataFrame
from scipy.optimize import curve_fit
import scipy.optimize as opt
from PIL import Image
import rasterio
from rasterio.plot import show
from pandas.plotting import register_matplotlib_converters
register_matplotlib_converters()
import shapely
from shapely.geometry import Point
from shapely.geometry import Polygon
from shapely.geometry import box
from shapely.geometry import shape
import zipfile
from pathlib import Path
```

If any package is not installed, the same can be installed in the jupyter-notebook itself, as

```
!conda install -c conda-forge <package name> --yes
                                    OR
!pip install <package name> --yes
```

12.3.2 Define the Directories for Processing

```
base_dir =Path('home/username/project')
hyp3_dir = Path(' home/username/project/data')
work_dir = Path(' home/username/project/output')
```

12.3.3 Unzipping the zipped files

If you have not unzipped the files as per section 8.1 then only do this step otherwise not.

```
zip_files = hyp3_dir.glob('*.zip')
for file in zip_files:
    try:
        with ZipFile(file, 'r') as f:
            f.extractall(hyp3_dir)
    except BadZipFile:
        print(file)
```

12.3.4 Clipping the Area of Interest (AOI)

If the extent of the downloaded images/interferograms is larger than the AOI, then AOI may be extracted from the following command.

```
# Clipping if input files are in GCS with WGS84
.....................................................
files = hyp3_dir.rglob('*.tif')
files_for_mintpy = ['_water_mask.tif', '_corr.tif',
'_conncomp.tif','_unw_phase.tif', '_wrapped_phase.tif','_dem.tif',
                    '_lv_theta.tif', '_lv_phi.tif']

for extension in files_for_mintpy:
        for file in hyp3_dir.rglob(f'*{extension}'):
            dst_file = file.parent / f'{file.stem}_clipped{file.suffix}'
            gdal.Warp(destNameOrDestDS=str(dst_file),
srcDSOrSrcDSTab=str(file),
                      outputBounds=[77.03, 28.53, 77.35, 28.74],
outputBoundsSRS='EPSG:4326', dstSRS='EPSG:4326')

# Clipping if input files are in UTM projection .......................
files = hyp3_dir.rglob('*.tif')
files_for_mintpy = ['_water_mask.tif', '_corr.tif', '_conncomp.tif',
'_unw_phase.tif', '_wrapped_phase.tif','_dem.tif', '_lv_theta.tif',
'_lv_phi.tif'] # if burst is downloaded

files_for_mintpy = ['_water_mask.tif', '_corr.tif', '_unw_phase.tif',
'_dem.tif', '_lv_theta.tif', '_lv_phi.tif'] # if whole slice is downloaded

for extension in files_for_mintpy:
        for file in hyp3_dir.rglob(f'*{extension}'):
            dst_file = file.parent / f'{file.stem}_clipped{file.suffix}'
            gdal.Warp(destNameOrDestDS=str(dst_file),
srcDSOrSrcDSTab=str(file),
                      # outputBounds=[xmin, ymin,xmax,ymax],
outputBoundsSRS='EPSG:326ZoneNumber', dstSRS='EPSG:326ZoneNumber')
                      outputBounds=[697848, 3158179, 729788, 3181067],
outputBoundsSRS='EPSG:32643', dstSRS='EPSG:32643')
```

12.3.5 Prepare the Template file for processing

```
data_dir = hyp3_dir

mintpy_config = Path(work_dir) / 'mintpy_config.txt'
mintpy_config.write_text(
f"""
# 1. load_data.............................................................
mintpy.load.processor        = hyp3
##---------interferogram datasets:
mintpy.load.unwFile         = {data_dir}/*/*_unw_phase_clipped.tif
mintpy.load.corFile         = {data_dir}/*/*_corr_clipped.tif
mintpy.load.connCompFile    = {data_dir}/*/*_conncomp_clipped.tif  #[path
pattern of connected components    files], optional but recommended
mintpy.load.intFile         = {data_dir}/*/*_wrapped_phase_clipped.tif
#[path pattern of wrapped interferogram   files], optional
```

```
mintpy.load.magFile         = auto  #[path pattern of interferogram magnitude
files], optional
##---------ionosphere stack (optional):
##---------offset stack (optional):
##---------geometry datasets:
mintpy.load.demFile          = {data_dir}/*/*_dem_clipped.tif
mintpy.load.incAngleFile     = {data_dir}/*/*_lv_theta_clipped.tif
mintpy.load.azAngleFile      = {data_dir}/*/*_lv_phi_clipped.tif
mintpy.load.waterMaskFile    = {data_dir}/*/*_water_mask_clipped.tif
##---------subset (optional):
## if both yx and lalo are specified, use lalo option unless a) no lookup
file AND b) dataset is in radar coord
mintpy.subset.yx     = auto    #[y0:y1,x0:x1 / no], auto for no
mintpy.subset.lalo   = auto    #[S:N,W:E / no], auto for no
##---------multilook (optional):

#2. modify_network.....................................................
## 1) Network modification based on temporal/perpendicular baselines, date,
num of connections etc.
## 2) Data-driven network modification
       ## a - Coherence-based network modification = (threshold + MST) by
default
       ## b - Effective Coherence Ratio network modification = (threshold +
MST) by default
mintpy.network.coherenceBased  = yes  #[yes / no], auto for no, exclude
interferograms with coh < minCoh
mintpy.network.minCoherence    = 0.40  #[0.0-1.0], auto for 0.7
mintpy.network.keepMinSpanTree = auto  #[yes / no], auto for yes, keep
interferograms in Min Span Tree network
mintpy.network.maskFile        = {work_dir}/waterMask.h5  #[file name, no],
auto for waterMask.h5 or no [if no waterMask.h5 found]
mintpy.network.aoiYX           = auto  #[y0:y1,x0:x1 / no], auto for no, area
of interest for coherence calculation
mintpy.network.aoiLALO         = auto  #[S:N,W:E / no], auto for no - use the
whole area

# 3. reference_point....................................................
## Reference all interferograms to one common point in space (any one option)
mintpy.reference.yx            = auto   #[257,151 / auto]
mintpy.reference.lalo          = auto   #[31.8,130.8 / auto]
mintpy.reference.maskFile      = {work_dir}/maskConnComp.h5   #[filename /
no], auto for maskConnComp.h5
mintpy.reference.coherenceFile = {work_dir}/avgSpatialCoh.h5   #[filename],
auto for avgSpatialCoh.h5
mintpy.reference.minCoherence  = 0.85    #[0.0-1.0], auto for 0.85, minimum
coherence for auto method

# 4. correct_unwrap_error (optional) ....................................
mintpy.unwrapError.method          = auto #[bridging / phase_closure /
bridging+phase_closure / no], auto for no
mintpy.unwrapError.waterMaskFile   = {work_dir}/waterMask.h5  #[waterMask.h5
/ no], auto for waterMask.h5 or no [if not found]
mintpy.unwrapError.connCompMinArea = auto  #[1-inf], auto for 2.5e3, discard
regions smaller than the min size in pixels
## phase_closure options:
```

```
mintpy.unwrapError.numSample       = auto  #[int>1], auto for 100, number of
samples to invert for common conn. comp.
## bridging options:
mintpy.unwrapError.ramp            = auto #[linear / quadratic], auto for no;
recommend linear for L-band data
mintpy.unwrapError.bridgePtsRadius = auto  #[1-inf], auto for 50, half size
of the window around end points

# 5. invert_network ..................................................
mintpy.networkInversion.weightFunc      = auto #[var / fim / coh / no], auto
for var
mintpy.networkInversion.waterMaskFile   = {work_dir}/waterMask.h5 #[filename
/ no], auto for waterMask.h5 or no [if not found]
mintpy.networkInversion.minNormVelocity = auto #[yes / no], auto for yes,
min-norm deformation velocity / phase
mintpy.networkInversion.maskDataset   = auto #[coherence / connectComponent /
rangeOffsetStd / azimuthOffsetStd / no], auto for no
mintpy.networkInversion.maskThreshold = auto #[0-inf], auto for 0.4
mintpy.networkInversion.minRedundancy = auto #[1-inf], auto for 1.0, min
num_ifgram for every SAR acquisition
mintpy.networkInversion.minTempCoh  = 0.3 #[0.0-1.0], auto for 0.7, min
temporal coherence for mask
mintpy.networkInversion.minNumPixel = auto #[int > 1], auto for 100, min
number of pixels in mask above
mintpy.networkInversion.shadowMask  = auto #[yes / no], auto for yes [if
shadowMask is in geometry file] or no.

# correct SET
mintpy.solidEarthTides = yes #[yes / no], auto for no

# 6. correct_troposphere (optional but recommended).....................
mintpy.troposphericDelay.method = height_correlation   #[pyaps /
height_correlation / gacos / no], auto for pyaps
## for pyapp............
mintpy.troposphericDelay.weatherModel = auto  #[ERA5 / MERRA / NARR], auto
for ERA5
mintpy.troposphericDelay.weatherDir   = auto  #[path2directory], auto for
WEATHER_DIR or "./"
## For height_correlation.........
mintpy.troposphericDelay.polyOrder      = auto  #[1 / 2 / 3], auto for 1
mintpy.troposphericDelay.looks          = auto  #[1-inf], auto for 8, extra
multilooking num
mintpy.troposphericDelay.minCorrelation = auto  #[0.0-1.0], auto for 0
## For GACOS ..............
mintpy.troposphericDelay.gacosDir = auto # [path2directory], auto for
"./GACOS"

# 7. deramp (optional)................................................
mintpy.deramp          = quadratic      #[no / linear / quadratic], auto for
no - no ramp will be removed
mintpy.deramp.maskFile = {work_dir}/maskTempCoh.h5 #[filename / no], auto for
maskTempCoh.h5, #mask file for ramp estimation

# 8. correct_topography (optional but recommended).....................
mintpy.topographicResidual           = auto  #[yes / no], auto for yes
```

```
mintpy.topographicResidual.polyOrder    = auto  #[1-inf], auto for 2, poly
order of temporal deformation model
mintpy.topographicResidual.phaseVelocity  = {work_dir}/avgPhaseVelocity.h5
#[yes / no], auto for no - use phase velocity for minimization
mintpy.topographicResidual.stepFuncDate   = auto  #[20080529,20190704T1733 /
no], auto for no, date of step jump
mintpy.topographicResidual.excludeDate   = auto  #[20070321 / txtFile / no],
auto for exclude_date.txt
mintpy.topographicResidual.pixelwiseGeometry =
{work_dir}/inputs/geometryGeo.h5  #[yes / no], auto for yes, use pixel-wise
geometry info

# 9.1 residual_RMS (root mean squares for noise evaluation).................
mintpy.residualRMS.maskFile = {work_dir}/maskTempCoh.h5  #[file name / no],
auto for maskTempCoh.h5, mask for ramp estimation
mintpy.residualRMS.deramp   = auto  #[quadratic / linear / no], auto for
quadratic
mintpy.residualRMS.cutoff   = auto  #[0.0-inf], auto for 3

# 9.2 reference_date.......................................................
# Reference all time-series to one date in time
mintpy.reference.date = auto   #[reference_date.txt / 20090214 / no], auto
for reference_date.txt

# 10. velocity.............................................................
mintpy.timeFunc.startDate   = auto   #[20070101 / no], auto for no
mintpy.timeFunc.endDate     = auto   #[20101230 / no], auto for no
mintpy.timeFunc.excludeDate = auto   #[exclude_date.txt / 20080520,20090817 /
no], auto for exclude_date.txt
## For a suite of Time Function
mintpy.timeFunc.polynomial = auto   #[int >= 0], auto for 1, degree of the
polynomial function
mintpy.timeFunc.periodic   = auto   #[1,0.5 / list_of_float / no], auto for
no, periods in decimal years
mintpy.timeFunc.stepDate   = auto   #[20110311,20170908 / 20120928T1733 /
no], auto for no, step function(s)
mintpy.timeFunc.exp        = auto   #[20110311,60 / 20110311,60,120 /
20110311,60;20170908,120 / no], auto for no
mintpy.timeFunc.log        = auto   #[20110311,60 / 20110311,60,120 /
20110311,60;20170908,120 / no], auto for no
## Uncertainity Quantification Method
mintpy.timeFunc.uncertaintyQuantification = auto   #[residue, covariance,
bootstrap], auto for residue
mintpy.timeFunc.timeSeriesCovFile         = auto   #[filename / no], auto for
no, time series covariance file
mintpy.timeFunc.bootstrapCount            = auto   #[int>1], auto for 400,
number of iterations for bootstrapping

# 11.1 geocode (post-processing): for input dataset in radar coordinates
only.............................
mintpy.geocode              = auto  #[yes / no], auto for yes
mintpy.geocode.SNWE         = auto  #[-1.2,0.5,-92,-91 / none ], auto for
none, output extent in degree
mintpy.geocode.laloStep     = auto  #[-0.000555556,0.000555556 / None], auto
for None, output resolution in degree
```

```
mintpy.geocode.interpMethod = auto  #[nearest], auto for nearest,
interpolation method
mintpy.geocode.fillValue    = auto  #[np.nan, 0, ...], auto for np.nan, fill
value for outliers.

# 11.2 google_earth (post-processing): Auto saved.....................
mintpy.save.kmz    = auto   #[yes / no], auto for yes, save geocoded velocity
to Google Earth KMZ file

# 11.3 hdfeos5 (post-processing) .....................................
mintpy.save.hdfEos5         = auto   #[yes / no], auto for no, save time-
series to HDF-EOS5 format
mintpy.save.hdfEos5.update  = auto   #[yes / no], auto for no, put XXXXXXXX
as endDate in output filename
mintpy.save.hdfEos5.subset  = auto   #[yes / no], auto for no, put subset
range info   in output filename

# 11.4 plot ..........................................................
mintpy.plot = auto  #[yes / no], auto for yes, plot files generated by
default processing to pic folder
mintpy.plot.dpi   = 400  #[int], auto for 150, number of dots per inch (DPI)
mintpy.plot.maxMemory = auto  #[float], auto for 4, max memory used by one
call of view.py for plotting.

from mintpy.utils import utils as ut
plot as pp
""")
```

The detail information about the processing template can be found in the mintpy_config.txt file or smallbaselineApp.cfg file under work_dir directory.

12.3.6 Run SBAS-InSAR Time-Series Application

There are two ways to process SBAS-InSAR in MintPy. The first one is to give a single command to process all SBAS-InSAR in sequence automatically (Non-Stop) and processing will end once all steps are finished. The processing can be done by running this command-

```
! smallbaselineApp.py --work-dir {work_dir} {configName}
```

For example:

```
! smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt
```

Second way of SBAS-InSAR processing in MintPy is to process all the steps one-by-one. For this, we have to give the name of processing step by the following two ways-

i) Process a specific step

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
<name of step>
```

For example:

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
load_data
```

ii) Process a specific set of steps

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --
startstep <name of start step> --stopstep <name of end step>
```

For example:

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --
startstep load_data --stopstep quick_overview
```

The more detailed usage can be found in help-

```
!smallbaselineApp.py --help
```

The sequence of SBAS-InSAR MintPy steps is: [‘load_data’, 'modify_network', 'reference_point', 'quick_overview', 'correct_unwrap_error', 'invert_network', 'correct_LOD', 'correct_SET', 'correct_ionosphere', 'correct_troposphere', 'deramp', 'correct_topography', 'residual_RMS', 'reference_date', 'velocity', 'geocode', 'google_earth', 'hdfeos5']

In this tutorial, the second way of processing is opted, and explained as-

Step 1: Load the stack of Interferogram into MintPy

This step loads the stack of interferograms (unwrapped phase, coherence, and/or connected components) and geometry files (height, incidence/azimuth angle, lookup tables etc.) into HDF5 files with multiple datasets and attributes.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
load_data
```

By running this command, the "inputs" directory inside the work directory is created and two HDF5 files i.e. *ifgramStack.h5* (contains 6 dataset cubes and multiple metadata: unwrapPhase, coherence, connectComponent, date, bperp, dropIfgram) and *geometryGeo.h5* (contains geometrical datasets including height, incidence angle, azimuth angle, shadow layover mask, slant range distance, and/or water mask).

If you want to see the content inside each .h5 file, run the following commands and explore-

```
!info.py ./output/inputs/geometryGeo.h5
!info.py ./output/inputs/ifgramStack.h5
!info.py ./output/inputs/geometryGeo.h5 --compact
!info.py ./output/inputs/ifgramStack.h5 –date
```

Plot all data related with one interferometric pair (yyyymmdd_yyyymmdd):

```
view.main('./output/inputs/ifgramStack.h5 yyyymmdd_yyyymmdd --ncols
3'.split())
```

Plotting the datasets of geometryGeo.h5 file (IncidenceAngle, azimuthAngle, slantRangeDistance, height):

```
%matplotlib inline
# view.py options
opt = ' -d ./output/inputs/geometryGeo.h5 -c jet_r --coord radar --scalebar
0.2 0.15 0.04 --ref-marker k+ --ref-size 10 --fontsize 12 --cbar-nbins 5 --
lalo-fs 12 --notitle --ylabel-rot 0 --noscalebar --nomultilook --noverbose --
dpi 400 --save'
cmd_list = [
    f'view.py ./output/inputs/geometryGeo.h5 incidenceAngle {opt} --nocbar --
lalo-loc 1 0 0 1',
```

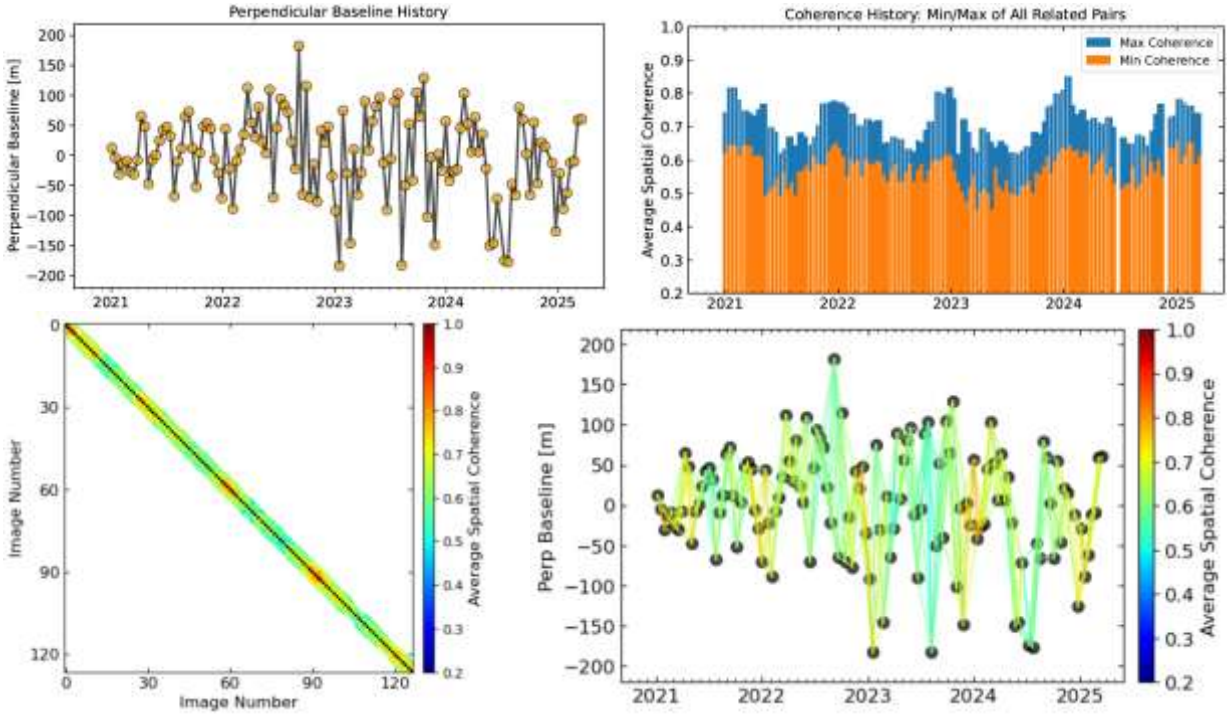
```
    f'view.py ./output/inputs/geometryGeo.h5 azimuthAngle {opt} --nocbar --
lalo-loc 0 0 0 1',
    f'view.py ./output/inputs/geometryGeo.h5 height {opt} --lalo-loc 0 0 0 1'
]

fig, axs = plt.subplots(nrows=1, ncols=3, figsize=[15, 10])
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)
fig.tight_layout()
points',ha='center',va='bottom',color='red',rotation=0)
plt.show()
```

Running plot_network.py gives an overview of the history of perpendicular baselines, average spatial coherence, coherence matrix, and network of interferograms.

```
plot_network.main('./output/inputs/ifgramStack.h5 -d coherence --show-kept -t
./output/mintpy_config.txt --notitle --mc black --ms 7 --fs 6 --lw 1 -c jet -
-figsize 6 4 --save'.split())
```

The plots will look like this-



Step 2: Modify the Interferogram Network (Optional)

This step (optional) modifies the network of interferograms based on average coherence, temporal and spatial baselines threshold, or by manual removal of specific pairs. This step helps in droping the interferograms with low coherence (mitigate the impact of decorrelation noise), and showing phase unwrapping errors (which could not be corrected for successfully).

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
modify_network
```

Before inversion, it's useful to take a look at the network of interferograms. Running plot_network.py gives an overview of the network and the average coherence of the stack.

```
plot_network.main('./output/inputs/ifgramStack.h5 -d coherence --show-kept –t
./output/mintpy_config.txt --notitle --mc black --ms 7 --fs 6 --lw 1 -c jet -
-figsize 6 4 --save'.split())
                                        OR

plot_network.main(['./output/inputs/ifgramStack.h5'])
```

Running this command creates multiple files as follows:

- *coherenceSpatialAvg.txt:* A simple text file that provides an overview to the stack and contains the interferogram dates, average coherence, temporal and spatial baseline separation.
- *coherenceMatrix.pdf:* Shows the average coherence pairs between all available pairs in the stack.
- *coherenceHistory.pdf:* Shows for each acquisition, the minimum and maximum average coherence among all pairs involving this acquisition. Useful to identify bad acquisitions, which usually have very low value for the maximum coherence. Those acquisitions should be dropped.
- *network.pdf*: Displays the network of interferograms on time-baseline coordinates, colorcoded by average coherence of the interferograms. Circles represent the acquisition dates and lines represent the interferograms. Solid lines are the interferograms used for time-series analysis and dashed line are the interferograms ignored in the time-series analysis.

Generate Average Coherence: Before running the time-series inversion, one may want to looks at average coherence in the stack. To create a map of average spatial coherence use temporal_average.py:

```
!temporal_average.py    ./output/inputs/ifgramStack.h5    -d    coherence    -o
avgSpatialCoh.h5
```

Generate Mask: The common connected component mask indicates pixels with valid connected component value in all kept interferograms. This is also used to guide the reference point selection.

```
!generate_mask.py  ./output/inputs/ifgramStack.h5 --nonzero -o
maskConnComp.h5 --update
```

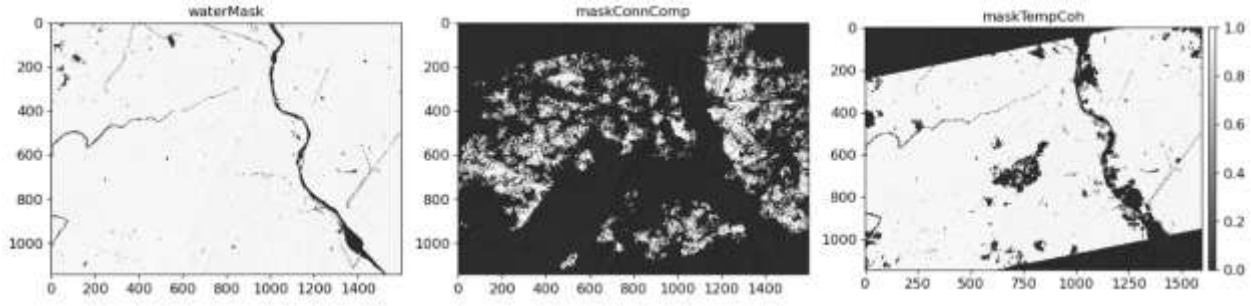
Plot the masks

```
%matplotlib inline
# view.py options
opt = ' -d ./output/inputs/geometryGeo.h5 -c gray –coord radar --scalebar 0.2
0.15 0.04 --ref-marker k+ --ref-size 10 --fontsize 12 --cbar-nbins 5 --lalo-
fs 12 --ylabel-rot 0 --nomultilook --noverbose --dpi 400 --save'
cmd_list = [
    f'view.py ./output/waterMask.h5 {opt} --nocbar --lalo-loc 1 0 0 1',
    f'view.py ./output/maskConnComp.h5 {opt} --nocbar --lalo-loc 0 0 0 1',
    f'view.py ./output/maskTempCoh.h5 {opt} --lalo-loc 0 0 0 1'
]

# plot using matplotlib & mintpy.view
# fig, axs = plt.subplots(nrows=1, ncols=3, figsize=[15, 10],
subplot_kw=dict(projection=ccrs.PlateCarree()))
# fig, axs = plt.subplots(nrows=1, ncols=3, figsize=[15, 10],
subplot_kw=dict(projection=ccrs.epsg()))
fig, axs = plt.subplots(nrows=1, ncols=3, figsize=[15, 10],
subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
```

```
    plot_slice(ax, data, atr, inps)
fig.tight_layout()
# plt.annotate("(d)",(lat,lon),xytext=(lat, lon),weight='normal',size=16,
textcoords='offset points',ha='center',va='bottom',color='red',rotation=0)
plt.show()
```

The masks plots are shown below-



This code can be used to save the figures

```
!view.py ./output/waterMask.h5 -v 0 1 --save
!view.py ./output/maskConnComp.h5 -v 0 1 -c jet --save
!view.py ./output/maskTempCoh.h5 -v 0 1 --save
```

Step 3: Select the spatial reference point

The unwrapped interferogram may be relative with respect to an arbitrary reference pixel. Therefore, this step introduces a common reference pixel to all interferograms. There are many approaches in mintpy to choose a reference pixel. There are some guidelines for selecting a reference point based on the prior knowledge of the study area (see template file).

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
reference_point
```

Running this step, adds additional attributes "REF_X, REF_Y" and "REF_LON, REF_LAT" to the ifgramStack.h5 file. To see the attributes of the file run info.py:

```
!info.py ./output/inputs/ifgramStack.h5 | egrep 'REF_'
```

Step 4: Assessing the quick overview about the parameters

This step provides a quick assessment of:

1. expected rate maps even before inversion by simply averaging / stacking the unwrapped interferograms.
2. distribution of unwrapping errors from the number of interferogram triplets with non-zero integer ambiguity.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
quick_overview
```

Plot the stacking result

```
view.main('avgPhaseVelocity.h5'.split())
```

For several stacks of interferograms, it's sometimes impractical to check every single interferogram for phase unwrapping errors. Below is a way for use one map to indicate potential phase unwrapping errors for

the entire stack. For an interferogram triplet $(\Delta\phi^{ij}, \Delta\phi^{jk}, and\ \Delta\phi^{ki})$, unwrapping errors will introduce a non-zero integer component C_{int}^{ijk} in the closure phase C^{ijk}. Therefore, the number of interferogram triplets with non-zero integer ambiguity T_{int} can be used to detect unwrapping errors.

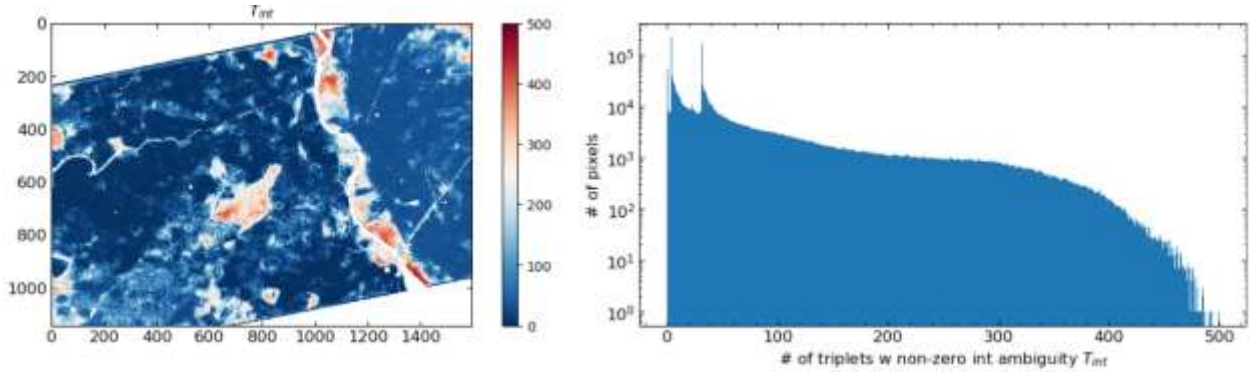
Explore the content of *numTriNonzeroIntAmbiguity.h5* file.

```
!info.py ./output/numTriNonzeroIntAmbiguity.h5 --compact
```

Plot the T_{int} as-

```
pp.plot_num_triplet_with_nonzero_integer_ambiguity('./output/numTriNonzeroInt
Ambiguity.h5', disp_fig=True, fig_size=[14, 4])
```

The plot will look like this-



Take home messages from T_{int} map and histogram:

1. Areas with Tint > 0 have unwrapping errors.
2. Areas share the common positive Tint value could be corrected.
3. Areas with wide-distributed Tint value indicates random unwrapping errors, which are difficult to correct.

Step 5: Correct Unwrapped Errors (Optional)

The input unwrapped interferograms may be affected by phase unwrapping errors (wrong integer number of 2π phase added during phase unwrapping). This step (optional) offers three methods to possibly correct unwrapping errors (See the template file).

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
correct_unwrap_error
```

Step 6: Invert the Network

This step inverts the stack of unwrapped interferograms to form the InSAR unwrapped phase time-series with respect to a reference acquisition date, which by default is the first acquisition. This is equivalent to transforming the network of small-baseline interferograms to a single-reference network of interferogram (unwrapped phase time series).

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
invert_network
```

The main product generated after running this step, is *timeseries.h5*. To see the general content of the file run info.py

```
!info.py ./output/timeseries.h5 --compact
```

The timeseries file contains three datasets i.e. interferometric 'phase time-series' for each acquisition relative to the reference acquisition, the 'date' for each acquisition, and the 'bperp' the timeseries of the perpendicular baseline. In addition to timeseries.h5, 'invert_network' step produces other quantities, which contain metrics to evaluate the quality of the inversion, e.g. temporalCoherence.h5 file. Temporal coherence represents the consistency between the timeseries and the network of interferograms. Pixels with temporal coherence values closer to 1 are considered reliable and pixels with temporal coherence values closer to zero are considered unreliable.

Plot the raw phase time-series and re-wrapped into [-5, 5) cm

```
view.main('./output/timeseries.h5 --wrap --wrap-range -5 5 -u cm --notitle --
notick --noaxis'.split())
```

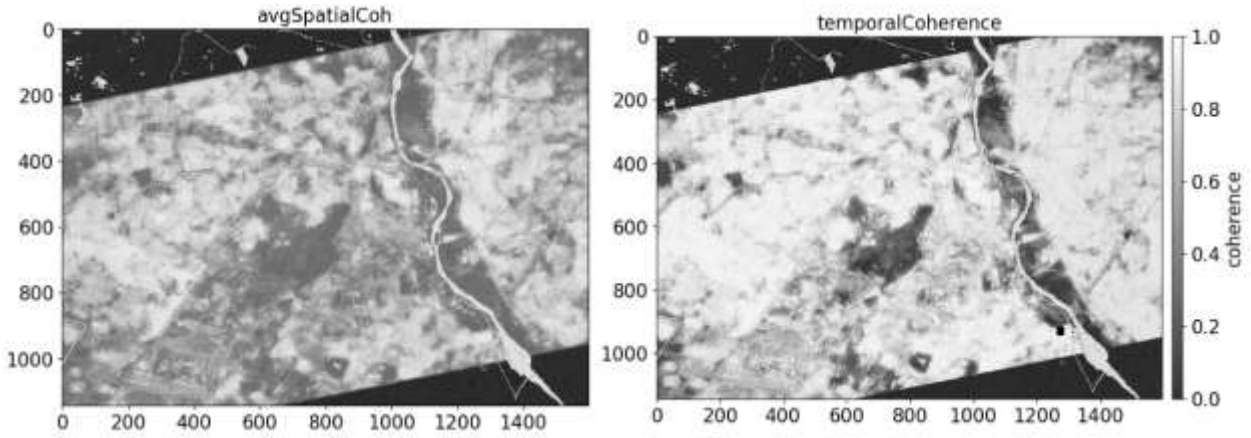
Plot the temporal coherence and mask temporal coherence-

```
view.main('./output/temporalCoherence.h5 -c gray --notick --noaxis --
noverbose'.split())

view.main('./output/maskTempCoh.h5 -c gray --notick --noaxis --
noverbose'.split())
```

Plot the average spatial and temporal coherence-

```
%matplotlib inline
opt = '-d ./output/inputs/geometryGeo.h5 -c grey -m ./output/waterMask.h5 --
cbar-label coherence -coord radar --noscalebar --cbar-nbins 5 --noverbose --
ylabel-rot 0 --save'
cmd_list = [
    f'view.py ./output/avgSpatialCoh.h5 {opt} --nocbar --lalo-loc 1 0 0 1',
    f'view.py ./output/temporalCoherence.h5 {opt} --lalo-loc 0 0 0 1'
]
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=[15, 10],
subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)
fig.tight_layout()
plt.show()
```



Now after network inversion, the noise reduction steps for phase time-series are processed.

Step 7: Correct Local Oscillator Drift (for ENVISAT only) (Optional)

This step is specific to Envisat data and applies an empirical correction to account for possible local oscillator drift of the radar.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
correct_LOD
```

Step 8: Correct Solid Earth Tides (SET)

This step corrects (optional) the solid Earth tides due to the gravitational pull of Sun and Moon.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
correct_SET
```

Plot the SET corrected time-series:

```
view.main('./output/timeseries_SET.h5 --wrap --wrap-range -5 5 --notitle --
notick --noaxis'.split())
```

Step 9: Correct Ionospheric Delay (Optional)

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
correct_ionosphere
```

Step 10: Correct Tropospheric Delay

It corrects tropospheric delay- i) using atmospheric models, or ii) with empirical phase elevation approach estimated from InSAR data.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
correct_troposphere
```

Plot the tropospheric corrected (height correlation method or ERA5) time-series:

```
view.main('./output/timeseries_SET_tropHgt(or ERA5).h5 --wrap --wrap-range -5
5 --notitle --notick --noaxis'.split())
```

Step 11: Removal of Ramp (optional)

This step (if requested) removes a ramp from each acquisition. Note that deramping removes residual long-wavelength interferometric phase components which may be due to noise (geometrical residual, atmospheric delay) or signal (tectonic deformation).

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
deramp
```

Plot time-series after deramp:

```
view.main('./output/timeseries_SET_tropHgt(or ERA5)_ramp.h5 --wrap --wrap-
range -5 5 --notitle --notick --noaxis'.split())
```

Step 12: Correct Topographic Error

It estimates residual topographic effects (due to DEM errors) which are correlated with temporal variation of perpendicular baseline.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
correct_topography
```

Plot the topographic corrected output:

```
view.main('./output/timeseries_SET_tropHgt(or ERA5)_ramp_demErr.h5 --wrap --
wrap-range -5 5 --notitle --notick --noaxis'.split())
```

This is the finally corrected output (cumulative displacement), and look like this figure below-

Step 13: Calculate RMS of Phase Residual

It estimates the average noise level for each acquisition by calculating the RMS of the residual phase.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
residual_RMS
```

Plot the RMS value

```
from mintpy.timeseries_rms import plot_rms_bar
txtContent = np.loadtxt('./output/rms_timeseriesResidual_ramp.txt',
dtype=bytes).astype(str)
rms_list = [float(i) for i in txtContent[:, 1]]
date_list = [i for i in txtContent[:, 0]]
fig, ax = plt.subplots(figsize=[10, 4])
ax = plot_rms_bar(ax, date_list, rms_list)
plt.show()
```

OR

Plot the same rms value as

```
pp.plot_timeseries_rms('./output/rms_timeseriesResidual_ramp.txt',
fig_size=[8, 4])
```

Step 14: Select Temporal Reference Date

It selects the reference date for time-series calculation.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
reference_date
```

Step 15: Estimate Velocity from time-series displacement

The ground deformation caused by many geophysical or anthropogenic processes are linear at first order approximation. Therefore, it is common to estimate the rate of the ground deformation which is the slope of linear fit to the time-series. This step estimates a suite of time functions, such as a linear velocity (rate of displacement).

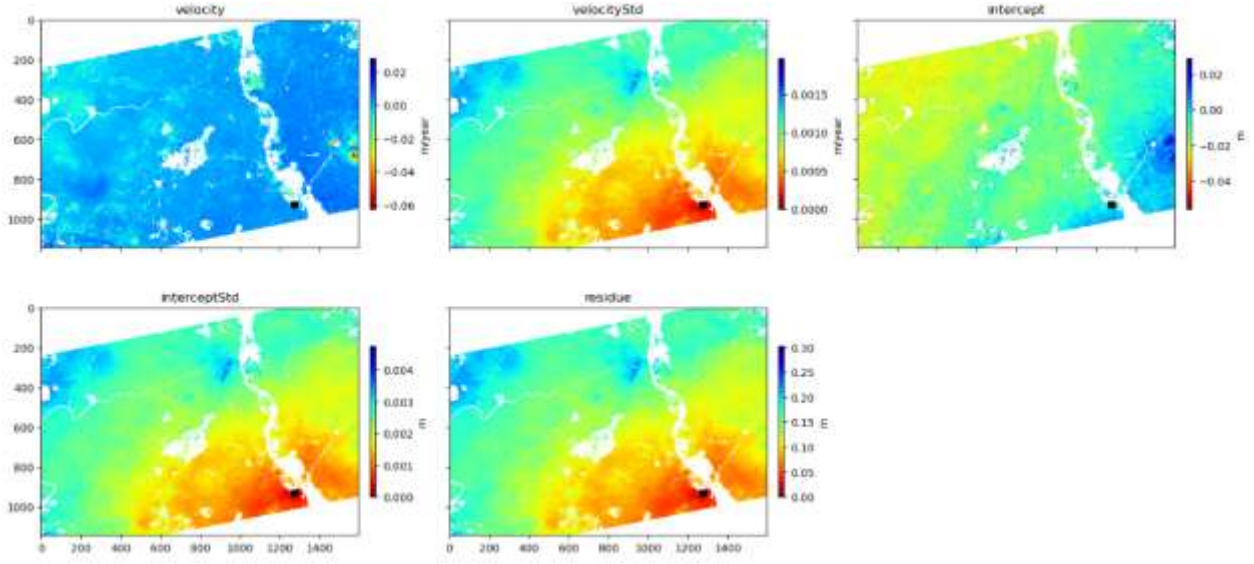
```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
velocity
```

See the content/dataset in velocity.h5 file.

```
!info.py ./output/velocity.h5 --compact
```

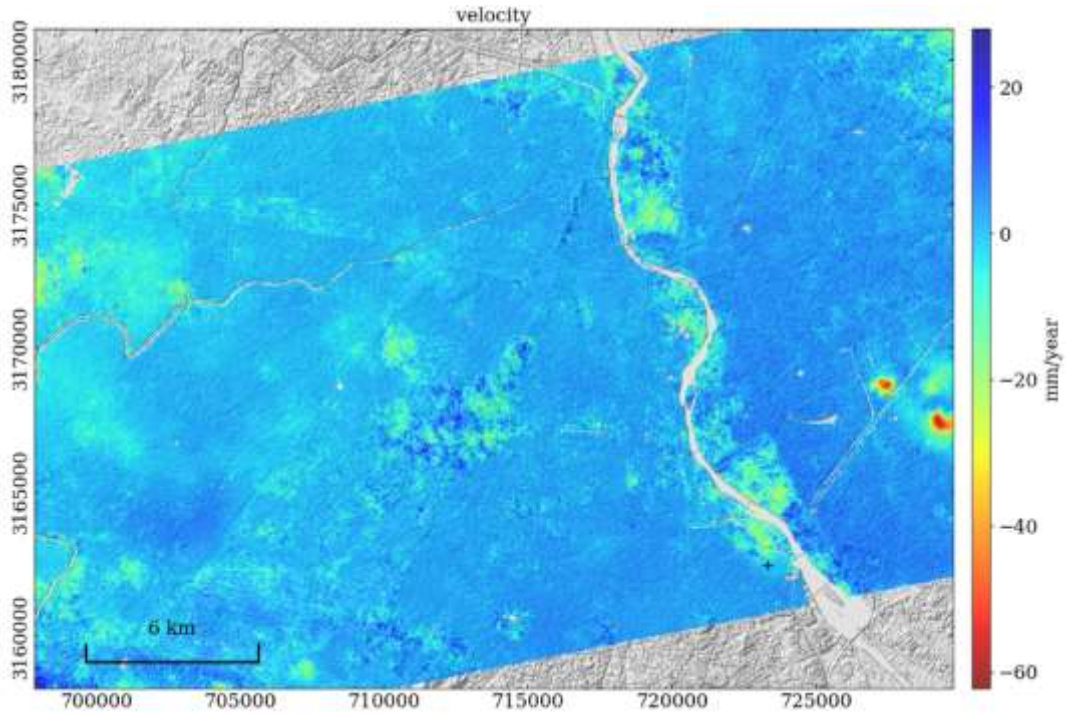
The estimated velocity also comes with an expression of uncertainty which is simply based on the goodness of fit while fitting a linear model to the time-series. This quantity is saved in "velocity.h5" under the velocityStd dataset. The standard deviation of velocity can be plotted as-

```
%matplotlib inline
view.main('./output/velocity.h5 -u mm -v 0 0.8'.split())
```



The velocity only in geocoordinate system with dem in the background and masked with water bodies, can be plotted as-

```
view.main('./output/velocity.h5 velocity -d ./output/inputs/geometryGeo.h5 -m
./output/waterMask.h5 --contour-step 5  --contour-lw 1 -u mm -c
jet_r'.split())
```



Calculating Time Function which includes velocity, acceleration, annual amplitude and annual phase information.

```
!timeseries2velocity.py ./output/timeseries_SET_tropHgt_ramp_demErr.h5 --poly
2 --periodic 1.0 0.5 --save-res -o ./output/velocityTimeFunc.h5
```

See the content/dataset in velocityTimeFunc.h5 file.

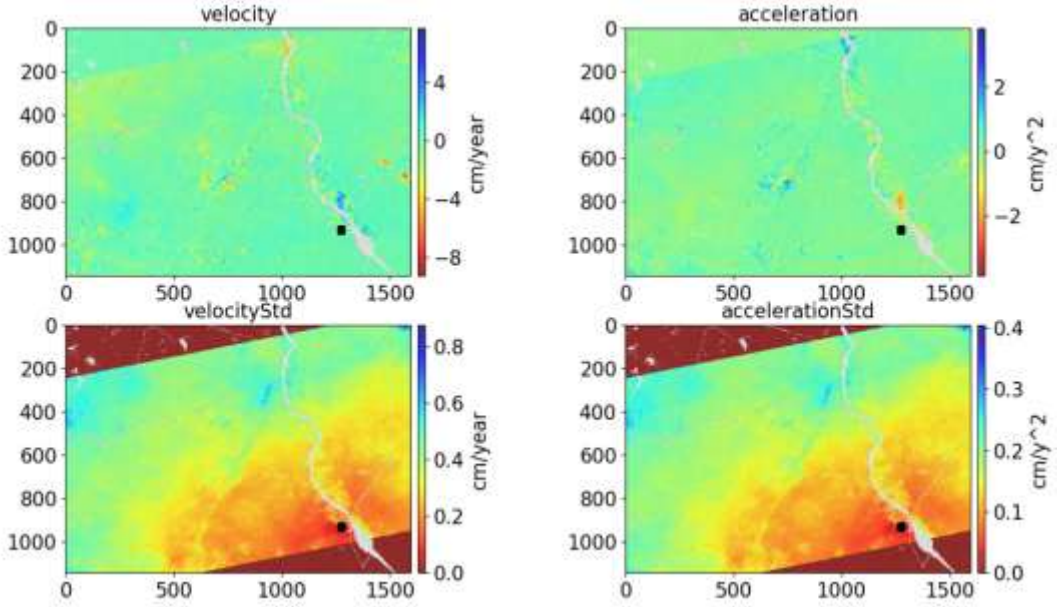
```
!info.py ./output/velocityTimeFunc.h5 --compact
```

Plot the velocityTimeFunc.h5

```
opt = '-d ./output/inputs/geometryGeo.h5 -c jet_r -m ./output/waterMask.h5 --
noscalebar --cbar-nbins 5 --noverbose --ylabel-rot 90 --save'
cmd_list = [
    f'view.py ./output/velocityTimeFunc.h5 velocity          {opt} --lalo-loc
1 0 0 0 ',
    f'view.py ./output/velocityTimeFunc.h5 acceleration      {opt} --lalo-loc
0 0 0 0 -u cm/y^2',
    f'view.py ./output/velocityTimeFunc.h5 velocityStd       {opt} --lalo-loc
1 0 0 1 ',
    f'view.py ./output/velocityTimeFunc.h5 accelerationStd   {opt} --lalo-loc
0 0 0 1 -u cm/y^2',
]

fig,      axs      =      plt.subplots(nrows=2,      ncols=2,      figsize=[11,
6],subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)
plt.show()
# plt.savefig('./Maps/TimeFuncGraphs.png', dpi=400,orientation='portrait')
```

The mean LOS velocity and its standard deviation were estimated using linear regression over InSAR LOS displacement time-series results. The mean LOS acceleration was estimated by the subtraction of the velocity at subsequent date from that of penultimate date and dividing by the time. The estimated mean LOS velocity, standard deviation of mean LOS velocity, mean LOS acceleration, and standard deviation of mean LOS acceleration are respectively plotted from above command.



Step 16: Geocode the Results

If the original stack in radar-coordinates, convert it to geo-coordinates in lat/lon. It's skipped if input dataset is in geo coordinates already.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
geocode
```

It outputs the following files into **./geo** folder:

- ./geo/geo_geometryRadar.h5
- ./geo/geo_maskTempCoh.h5
- ./geo/geo_temporalCoherence.h5
- ./geo/geo_timeseries_ERA5_ramp_demErr.h5
- ./geo/geo_velocity.h5

The above processed datasets are already in geocoded. Plot the geocoded velocity can be plotted as:

```
view.main('./output/velocity.h5 velocity --dem
./output/inputs/geometryRadar.h5 --shade-exag 0.05 --figsize 12 12 --notick -
-noaxis'.split())
```

Step 17: Export the results for Google Earth

This step generates the average velocity into a Google Earth KMZ file.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
google_earth
```

Step 18: Generate HDF-EOS5 file

The output displacement time-series with geometry info into one file in HDF-EOS5 format.

```
!smallbaselineApp.py --dir {work_dir} {work_dir}/mintpy_config.txt --dostep
hdfeos5
```

12.3.7 Visualization of Outputs/Results

All HDF5 files can be plotted using `view.py` and/or `tsview.py`. For plotting the InSAR results in MintPy, the following scripts may be used:

- `info.py` # check HDF5 file structure and metadata
- `view.py` # 2D map view
- `tsview.py` # 1D point time-series (interactive)
- `plot_coherence_matrix.py` # plot coherence matrix for one pixel (interactive)
- `plot_network.py` # plot network configuration of the dataset
- `plot_transection.py` # plot 1D profile along a line of a 2D matrix (interactive)
- `save_kmz.py` # generate Google Earth KMZ file in points or raster image
- `save_kmz_timeseries.py` # generate Google Earth KMZ file in points for time-series (interactive)

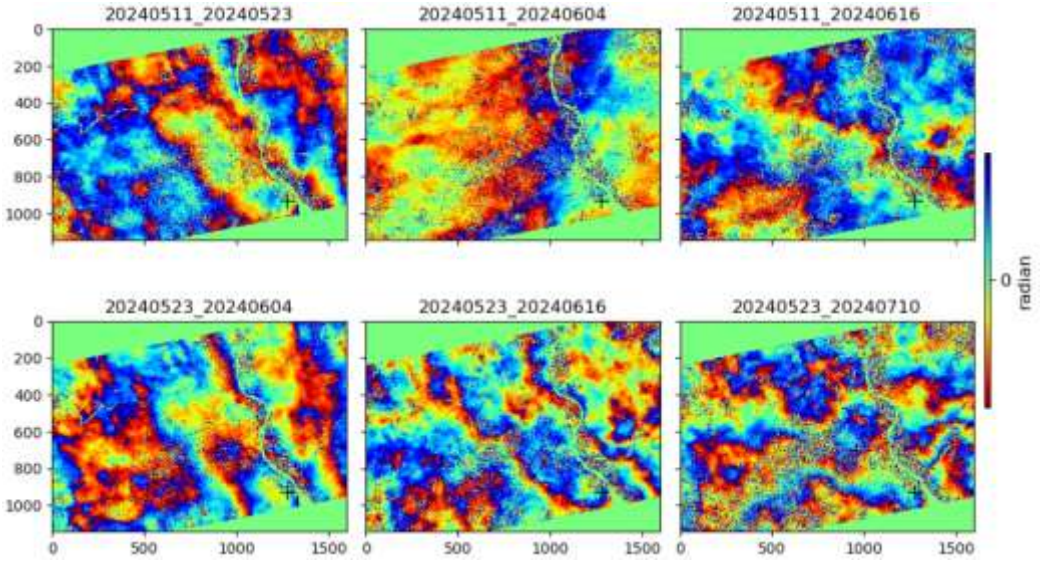
For more detailed documentation and processing parameters, one can visit the https://mintpy.readthedocs.io/en/latest/.

1) Plot the Wrapped and Unwrapped Interferograms:

Wrapped Interferograms.....

```
view.main('./output/inputs/ifgramStack.h5 unwrapPhase-202405* --wrap --wrap-range -3.14 3.14 -v -3.14 3.14 -d ./output/inputs/geometryGeo.h5 -c jet_r --show-kept --ref-marker k+ --ref-size 10 --noscalebar --alpha 1 --dpi 500 --noverbose --figsize 15 10 -o ./Maps/wrapPhase'.split())
```

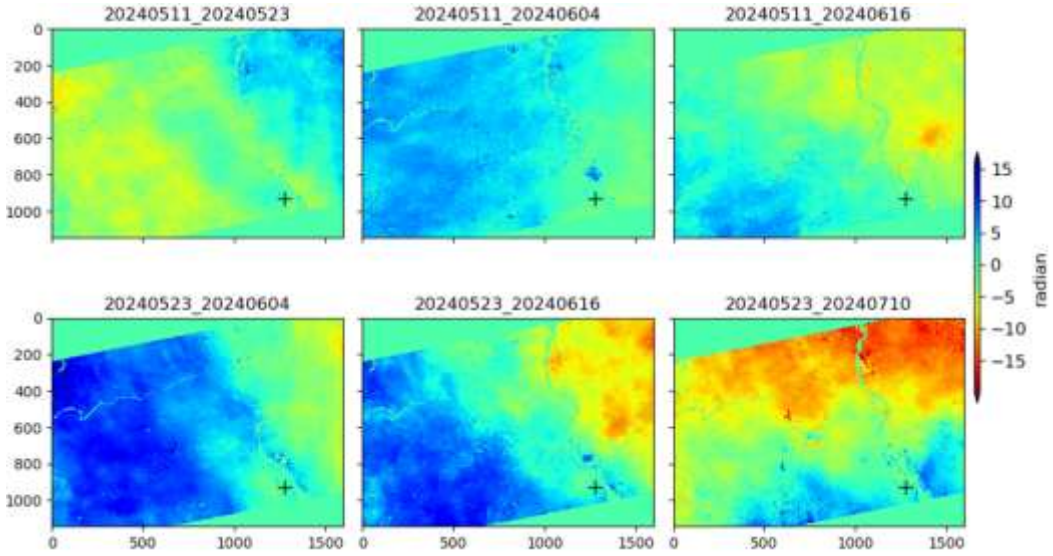
The wrapped interferogram which includes the acquisition(s) of May 2024, are plotted as



Unwrapped Interferograms.....

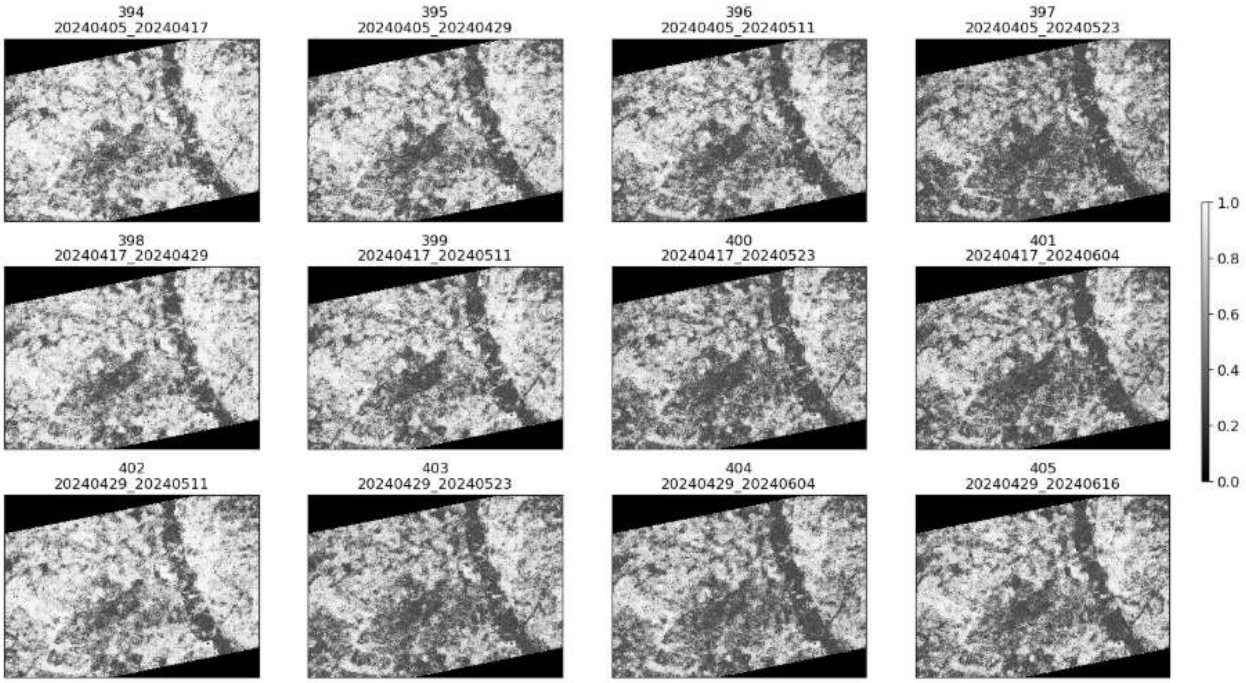
```
view.main('./output/inputs/ifgramStack.h5 unwrapPhase-202405* --cbar-nbins 8 -d ./output/inputs/geometryGeo.h5 -m ./output/waterMask.h5 -c jet_r --show-kept --ref-marker k+ --ref-size 10 --noscalebar --alpha 1 --dpi 500 --noverbose --figsize 15 10 -o ./Maps/UnwrapPhase'.split())
```

The unwrapped interferograms with the same dates are-



2) Plot spatial Coherence

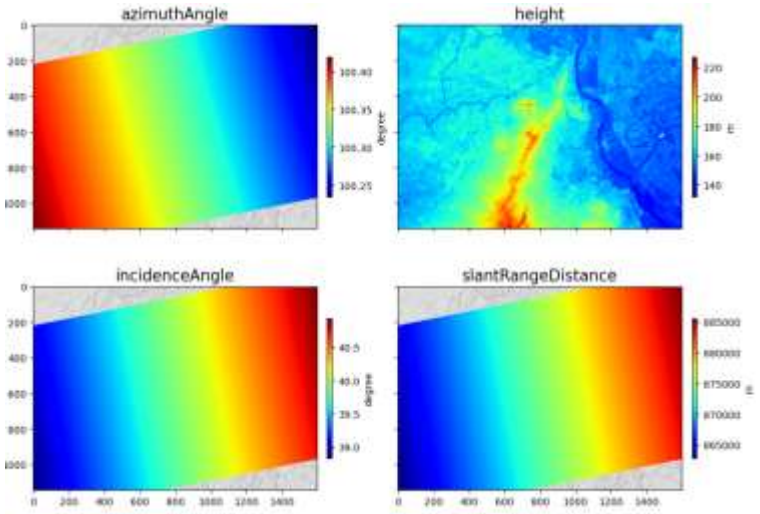
```
view.main('./output/inputs/ifgramStack.h5 coherence-202404* -v 0 1 --cbar-
nbins 6 -d ./output/inputs/geometryGeo.h5 -c gray --show-kept --ref-size 10 -
-noscalebar --alpha 1 --noverbose –o ./Maps/Coherence'.split())
```



3) Plot the geometry files

```
view.main('./output/inputs/geometryGeo.h5 incidenceAngle azimuthAngle height
slantRangeDistance -d ./output/inputs/geometryGeo.h5 --fontsize 16 -c jet --
scalebar 0.2 75 160 --alpha 1 --dpi 500 --pts-marker kd --pts-ms 8 --ref-marker
k+ --ref-size 6 --noverbose --figsize 10 8 -o ./Maps/Geometry_Files'.split())
```

The plots og geometry files (azimuth angle, height, incidence angle, and stant range distance) are-



4) Plot the inverted raw phase time-series and corrected time-series displacement

```
%matplotlib inline
# view.py options
opt = 'timeseries*2024 -d ./output/inputs/geometryGeo.h5 -m
./output/maskTempCoh.h5 --ref-date 20210103 --ref-marker k+ --ref-size 18 --
pts-marker kd --pts-ms 10 --fignum 6 --nrows 3 --wspace 5 --ncols 2 --
fontsize 24 --cbar-nbins 7 --lalo-fs 20 --figsize 25 25 --ylabel-rot 0 -c
jet_r --noverbose --dpi 400 -o ./Maps/Disp_Corrections'
cmd_list = [
    f'view.py ./output/timeseries.h5 {opt} --lalo-loc 1 0 1 0',
    f'view.py ./output/timeseries_SET.h5 {opt} --lalo-loc 0 1 1 0',
    f'view.py ./output/timeseries_SET_tropHgt.h5 {opt} --lalo-loc 1 0 0 0',
    f'view.py ./output/timeseries_SET_tropHgt_ramp.h5 {opt} --lalo-loc 0 1 0
0',
    f'view.py ./output/timeseries_SET_tropHgt_ramp_demErr.h5 {opt} --lalo-loc
1 0 0 1',
    f'view.py ./output/timeseriesResidual_ramp.h5 {opt} --lalo-loc 0 1 0 1'
]

# plot using matplotlib & mintpy.view
fig, axs = plt.subplots(nrows=3, ncols=2, figsize=[25, 25],
subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)
fig.tight_layout()
plt.show()
```

OR

```
%matplotlib inline
# tsview.py options
opt = ' timeseries*2024 --show-kept -d ./output/inputs/geometryGeo.h5 -m
./output/waterMask.h5 --coord radar --notitle --ref-date 20210103 --fontsize
24 --cbar-nbins 7 --lalo-fs 20 --figsize 15 10 --ylabel-rot 0 -c jet_r --
noverbose --dpi 400 -o ./Maps/Disp_Corrections'
cmd_list = [
    f'tsview.py ./output/timeseries.h5 {opt} --lalo-loc 1 0 1 0',
    f'tsview.py ./output/timeseries_SET.h5 {opt} --lalo-loc 1 0 0 0',
    f'tsview.py ./output/timeseries_SET_tropHgt.h5 {opt} --lalo-loc 1 0 0 1',
    f'tsview.py ./output/timeseries_SET_tropHgt_demErr.h5 {opt} --lalo-loc 0
1 0 1'
]

fig, axs = plt.subplots(nrows=2, ncols=2, figsize=[15,
10],subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)

fig.tight_layout()
plt.show()
```

5) Read & plot displacement time-series of one pixel from time-series HDF5 file

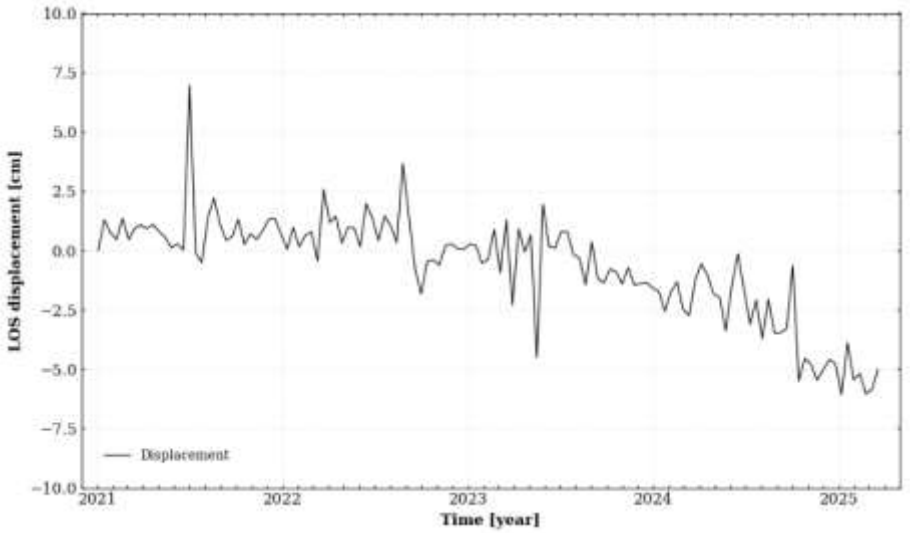
```
## Read displacement time-series of one pixel from timeseries HDF5 file
plt.rcParams.update({'font.size': 12})
geom_file = None
dts_file = os.path.join(base_dir, './output/timeseries.h5')

# read dates and displacement at selected point (lat, lon)

p1=[3168972.77, 729258]
dates, dis, dis_std = ut.read_timeseries_lalo(p1[0], p1[1], ts_file=dts,
lookup_file=geom_file)

# Scatter plot at the selected point
fig, ax = plt.subplots(nrows=1, ncols=1, figsize=[6, 3])
ax.scatter(dates, dis*100, marker='d', s=4**2, facecolors='red',
edgecolors='r', linewidth=1.)
ax.plot(dates, dis*100, color='red', linewidth=1.)
# axis format
ax.tick_params(which='both', direction='in', bottom=True, top=True,
left=True, right=True)
pp.auto_adjust_xaxis_date(ax, dates)
ax.set_ylim(-10, 10)
ax.set_xlabel('Time [year]')
ax.set_ylabel('LOS displacement [cm]')
fig.tight_layout()

# save and show the output plot
plt.savefig('./Maps/TS-Single_Point.png', dpi=300, orientation='portrait',
bbox_inches='tight',transparent=True)
plt.show()
```



6) Read & plot displacement time-series and subsequent corrected displacement time-series of one pixel from time-series HDF5 files

```
## Read displacement time-series and displacement time-series after
correction of one pixel from timeseries HDF5 files
```

```
plt.rcParams.update({'font.size': 12})
geom_file = None
dts = os.path.join(base_dir, './output/timeseries.h5')
dts_SET = os.path.join(base_dir, './output/timeseries_SET.h5')
dts_SET_tropHgt = os.path.join(base_dir,
'./output/timeseries_SET_tropHgt.h5')
dts_SET_tropHgt_demErr = os.path.join(base_dir,
'./output/timeseries_SET_tropHgt_demErr.h5')

# read dates and displacement at selected point (lat, lon)
p1=[3168972.77, 729258]
dates1, dis1, dis_std1 = ut.read_timeseries_lalo(p1[0], p1[1], ts_file=dts,
lookup_file=geom_file)
dates2, dis2, dis_std2 = ut.read_timeseries_lalo(p1[0], p1[1],
ts_file=dts_SET, lookup_file=geom_file)
dates3, dis3, dis_std3 = ut.read_timeseries_lalo(p1[0], p1[1],
ts_file=dts_SET_tropHgt, lookup_file=geom_file)
dates4, dis4, dis_std4 = ut.read_timeseries_lalo(p1[0], p1[1],
ts_file=dts_SET_tropHgt_demErr, lookup_file=geom_file)

# Scatter plots at the selected point
fig, ax = plt.subplots(nrows=1, ncols=1, figsize=[10, 6])
# ax.scatter(dates1, dis1*100, marker='x', s=4**2, facecolors='red',
edgecolors='r', linewidth=1.)
# ax.scatter(dates2, dis2*100, marker='o', s=4**2, facecolors='green',
edgecolors='g', linewidth=1.)
# ax.scatter(dates3, dis3*100, marker='d', s=4**2, facecolors='blue',
edgecolors='b', linewidth=1.)
# ax.scatter(dates4, dis4*100, marker='s', s=4**2, facecolors='cyan',
edgecolors='c', linewidth=1.)

ax.plot(dates1, dis1*100, color='black', linewidth=1.,alpha=1, linestyle='-',
label='Displacement')
ax.plot(dates2, dis2*100, color='green', linewidth=1.,alpha=1, linestyle='-',
label='SET Corrected Displacement')
ax.plot(dates3, dis3*100, color='blue', linewidth=1.,alpha=1, linestyle='-',
label='SET and Tropospheric Delay Corrected Displacement')
ax.plot(dates4, dis4*100, color='red', linewidth=1.,alpha=1, linestyle='-',
label='SET, Tropospheric Delay, and Topography Corrected Displacement')

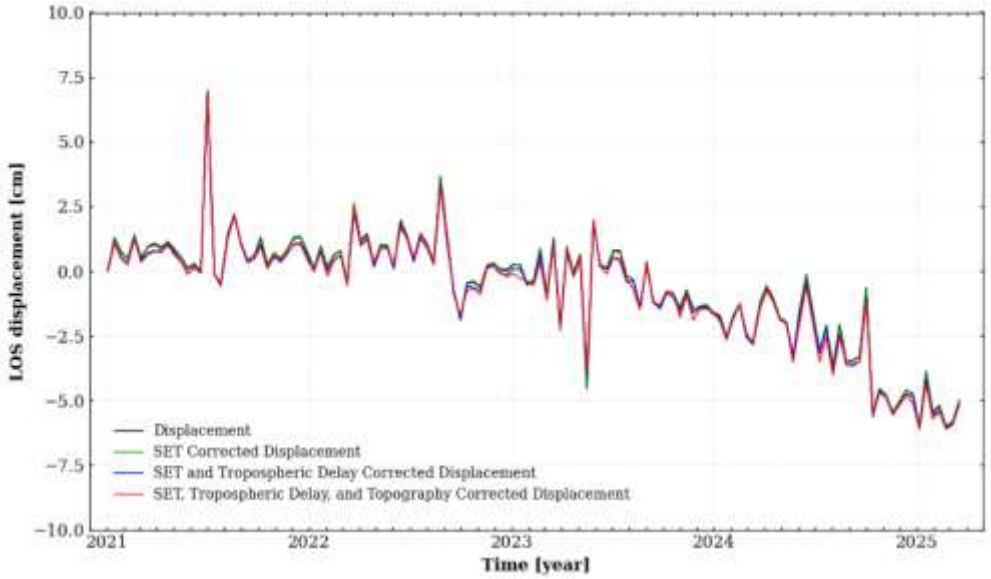
# axis format
ax.tick_params(which='both', direction='in', bottom=True, top=True,
left=True, right=True,length=2, width=1, color="black", labelsize=12,)
pp.auto_adjust_xaxis_date(ax, dates)
ax.set_xlim([datetime.date(2020,12,1), datetime.date(2025,5,1)])
ax.set_ylim(-10, 10)
ax.set_xlabel('Time [year]',fontweight="bold", fontsize=12)
ax.set_ylabel('LOS displacement [cm]',fontweight="bold", fontsize=12)
ax.legend(loc=(0.02,0.04),frameon=False, fontsize=10, ncol=1, framealpha=1,
shadow=False, borderpad=0.5,labelspacing=0.5,title='')
ax.grid(axis="x", color="black", alpha=.1, linewidth=1, linestyle=":")
ax.grid(axis="y", color="black", alpha=.1, linewidth=1, linestyle=':')

ax.spines['left'].set_color('k')
ax.spines['right'].set_color('k')
ax.spines['top'].set_color('k')
```

```
ax.spines['bottom'].set_color('k')
plt.rcParams['font.family'] = 'DeJavu Serif'
plt.rcParams['font.serif'] = ['Times New Roman']

# ax.annotate('P1 =
(%.5f,%.5f)'%(p1[0],p1[1]),(mdates.date2num(TsShp11_x[4]),TsShp11_y[2]),
#                   xytext=(80, -50),size=14,textcoords='offset
points',ha='center',va='bottom',color='black',rotation=0)

# save and show the output plot
fig.tight_layout()
plt.savefig('./Maps/TS-Correction_Single_Point.png', dpi=300,
orientation='portrait', bbox_inches='tight',transparent=True)
plt.show()
```

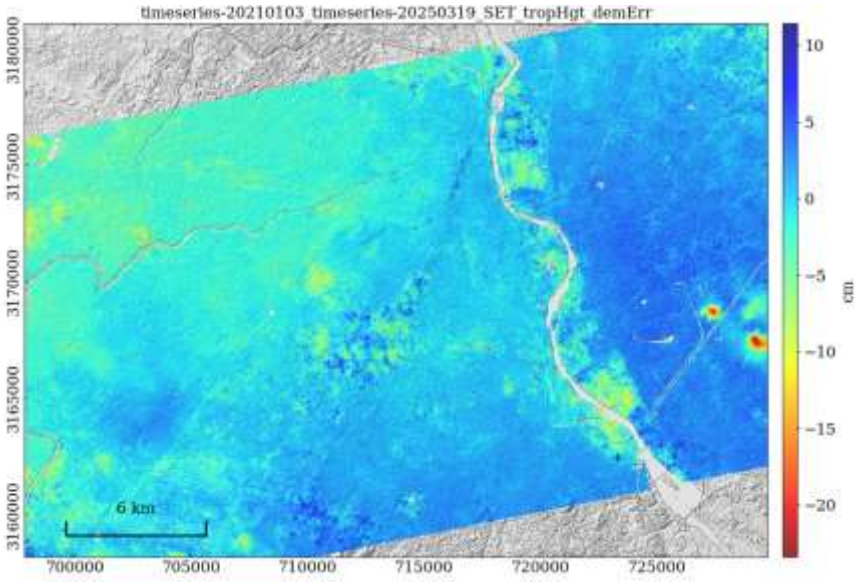


7) Plotting the cumulative LOS displacement, Mean LOS Velocity, and Mean LOS Acceleration

```
# Cumulative LOS displacement time-series
view.main('./output/timeseries_SET_tropHgt_demErr.h5 --ylabel-rot 90 -u cm --
ref-date 20210103 --ref-marker k+ --ref-size 8 --nomultilook -d
./output/inputs/geometryGeo.h5 --shade-exag 2 --noverbose -c jet_r --dpi 500
--figsize 15 10 --lalo-fs 10 --contour-smooth 5 --save -o
./Maps/Cummu_disp.png'.split())
```

OR

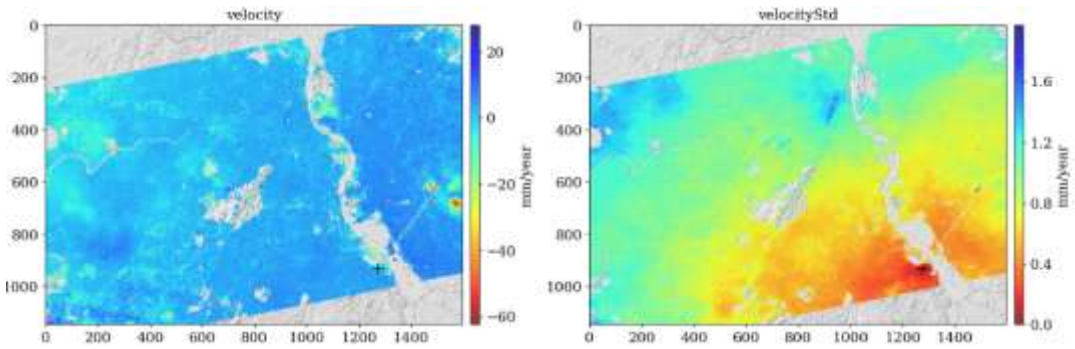
```
tsview.main('./output/timeseries_SET_tropHgt_demErr.h5  --animation  --ylabel-
rot 90 -u cm --ref-date 20210103 --ref-marker k+ --ref-size 8 --nomultilook -d
./output/inputs/geometryGeo.h5 --shade-exag 2 --noverbose -c jet_r --poly 2 --
periodic  1.0  --dpi  500  --figsize  25  25  --contour-smooth  5  --save  -o
./Maps/TS_Com_disp.png'.split())
```




```
# Mean LOS velocity and its standard deviation.....................
# From velocity.h5.....................

%matplotlib inline
opt = ' -d ./output/inputs/geometryGeo.h5 -m ./output/maskTempCoh.h5 --pts-
marker kd --pts-ms 8 --coord radar --ref-marker k+ --ref-size 12 --fignum 2 -
-fontsize 14 --cbar-nbins 5 --lalo-label --lalo-fs 12 --ylabel-rot 0 --
noscalebar -c jet_r --nomultilook --noverbose --dpi 400 -o
./Maps/VelocityGraphs'
cmd_list = [
    f'view.py ./output/velocity.h5 velocity {opt} -u mm --lalo-loc 1 0 1 0',
    f'view.py ./output/velocity.h5 velocityStd {opt} -u mm --lalo-loc 0 0 1
0']

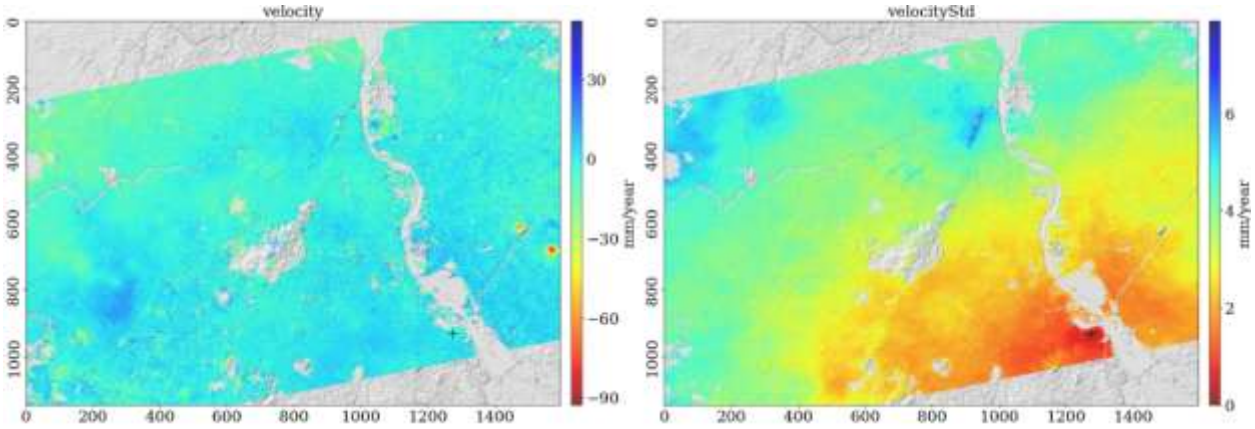
# fig, axs = plt.subplots(nrows=1, ncols=2, figsize=[15, 10],
subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=[15, 10])
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)
fig.tight_layout()
plt.show()
# plt.savefig('TimeFuncGraphs.png')
```



```
# From velocityTimeFunc.h5 ..............................
%matplotlib inline
opt = ' -d ./output/inputs/geometryGeo.h5 -m ./output/maskTempCoh.h5 --coord
radar --ref-marker k+ --ref-size 14 --fontsize 22 --cbar-nbins 5 --ylabel-rot
90 --noscalebar -c jet_r --nomultilook --noverbose --dpi 400 -o
./Maps/TimeVelocityGraphs'
cmd_list = [
    f'tsview.py ./output/velocityTimeFunc.h5 velocity {opt} -u mm/y',
    f'tsview.py ./output/velocityTimeFunc.h5 velocityStd {opt} -u mm/y']

# plot using matplotlib & mintpy.view
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=[25, 15])
# fig, axs = plt.subplots(nrows=1, ncols=2, figsize=[15, 10],
subplot_kw=dict(projection=ccrs.UTM(43,southern_hemisphere=False,
globe=None)))
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)

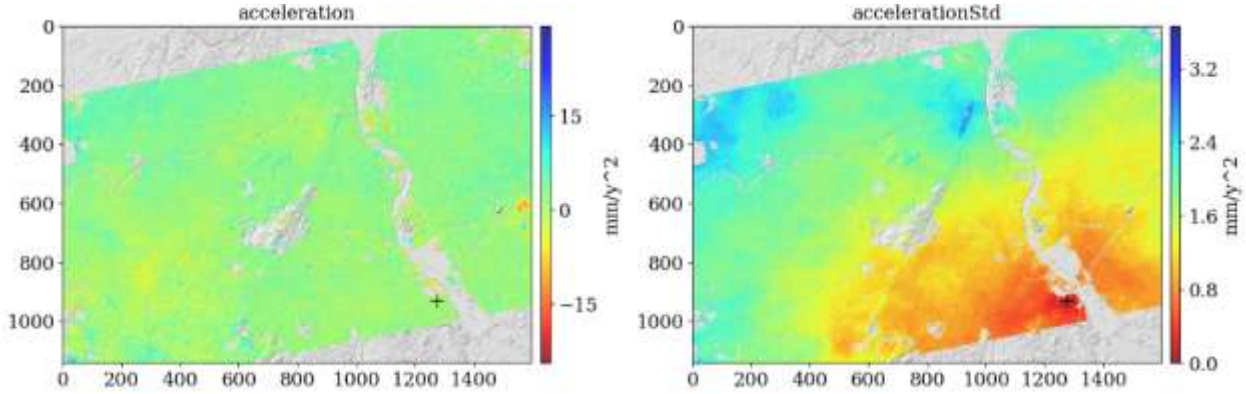
fig.tight_layout()
plt.show()
plt.savefig('./Maps/TimeFuncGraphs.png', dpi=400,orientation='portrait')
```




```
# Mean LOS acceleration and its standard deviation.....

# Only Possible from velocityTimeFunc.h5.....................
%matplotlib inline
# view.py options
opt = ' -d ./output/inputs/geometryGeo.h5 -m ./output/maskTempCoh.h5 --
scalebar 0.2 0.15 0.04 --coord radar --pts-marker kd --pts-ms 8 --ref-marker
k+ --ref-size 12 --fignum 2 --nrows 1 --ncols 2 --fontsize 14 --cbar-nbins 5
--lalo-label --lalo-fs 12 --notitle --ylabel-rot 0 --noscalebar -c jet_r --
nomultilook --noverbose --dpi 400 -o ./Maps/VelocityGraphs'
cmd_list = [
    f'view.py ./output/velocityTimeFunc.h5 acceleration {opt} -u mm/y^2 --
lalo-loc 1 0 0 1',
    f'view.py ./output/velocityTimeFunc.h5 accelerationStd {opt} -u mm/y^2 --
lalo-loc 0 0 0 1'
]
fig, axs = plt.subplots(nrows=1, ncols=2, figsize=[15, 10])
```

```
for ax, cmd in zip(axs.flatten(), cmd_list):
    data, atr, inps = prep_slice(cmd)
    plot_slice(ax, data, atr, inps)
fig.tight_layout()
plt.show()
# plt.savefig('./Maps/Acceleration.png', dpi=400,orientation='portrait')
```

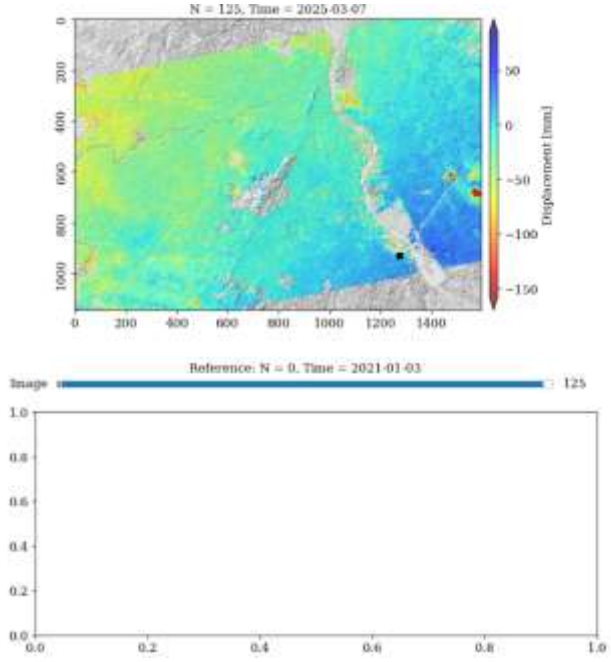


8) Time-Series Plot for multiple files

```
# This can be done in two ways (Jupyter-notebook and Terminal)

## In Jupyter-notebook

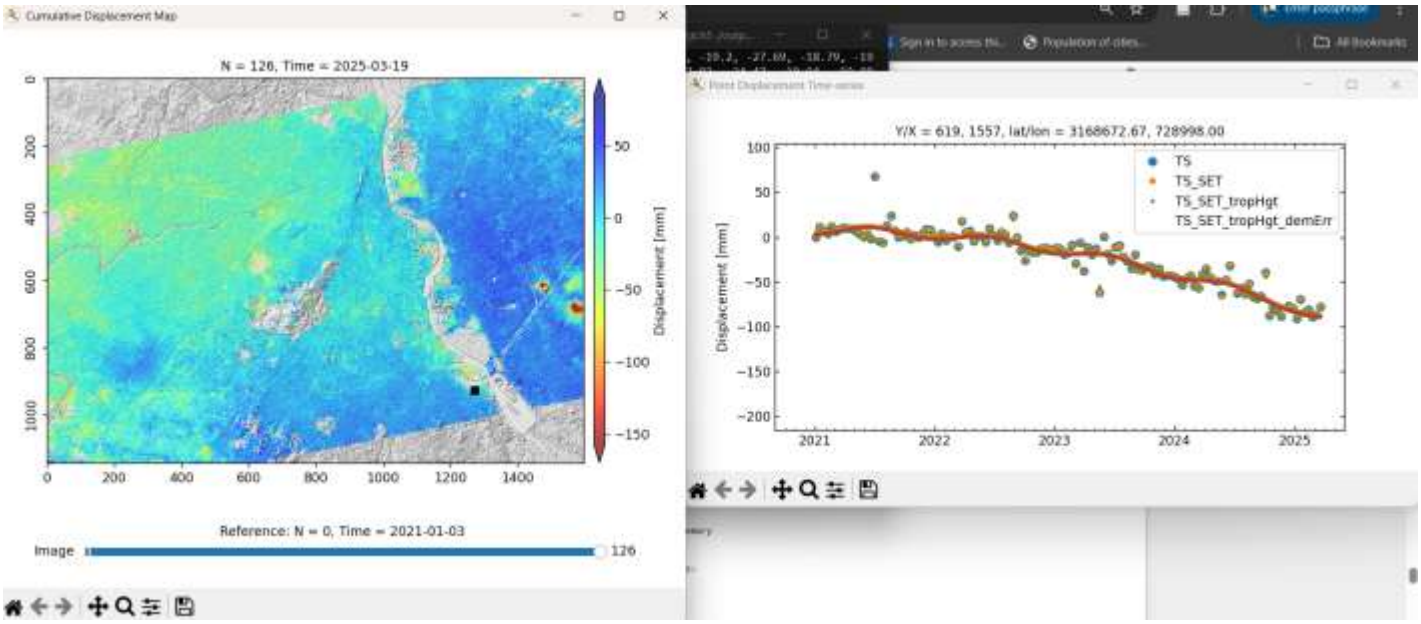
tsview.main('./output/timeseries_SET_tropHgt_ramp_demErr.h5
./output/timeseries_SET_tropHgt_ramp.h5 ./output/timeseries_SET_tropHgt.h5
./output/timeseries_SET.h5 ./output/timeseries.h5 -d
./output/inputs/geometryGeo.h5 --shade-exag 2 --coord radar --ylabel-rot 90 -
u mm --ref-date 20210103 -c jet_r --poly 2 --periodic 1.0 --dpi 500 --figsize
10 4 --contour-smooth 5 --figsize 8 3 --scalebar 0.2 0.15 0.04 --save -o
./Maps/TS_graphs.png'.split())
```



```
## In Terminal

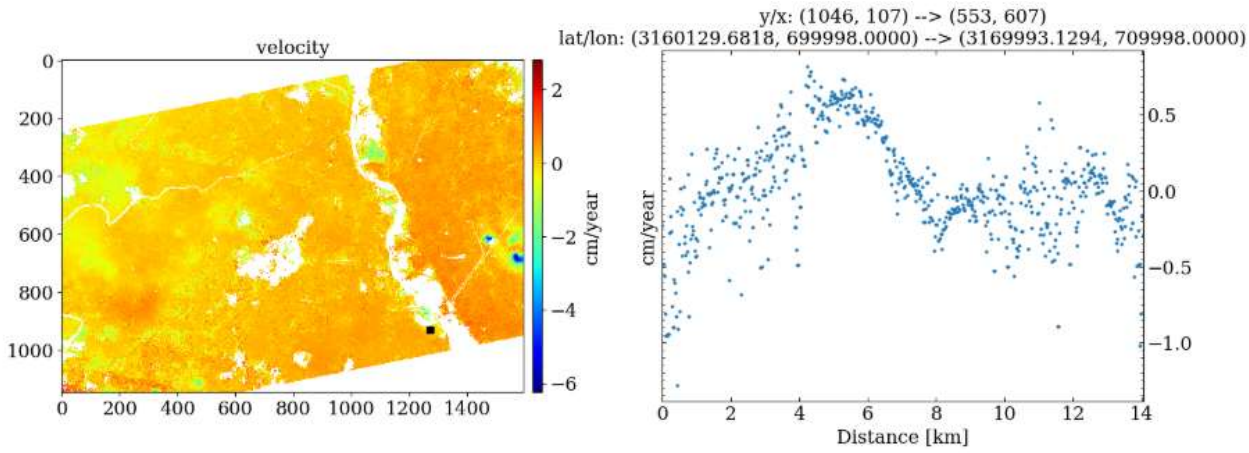
tsview.py ./output/timeseries_SET_tropHgt_demErr.h5
./output/timeseries_SET_tropHgt.h5 ./output/timeseries_SET.h5
./output/timeseries.h5 -d ./output/inputs/geometryGeo.h5 --shade-exag 2 --
coord radar --ylabel-rot 90 -u mm --ref-date 20210103 -c jet_r --poly 2 --
periodic 1.0 --dpi 500 --figsize 10 4 --contour-smooth 5 --figsize 8 3 --
scalebar 0.2 0.15 0.04 –save
```

A new window will prompt and you have to select a point on the map to plot its time-series. The scrollbar (at the bottom of the map) can also be used to see the spatio-temporal profile of the deformation on the map.

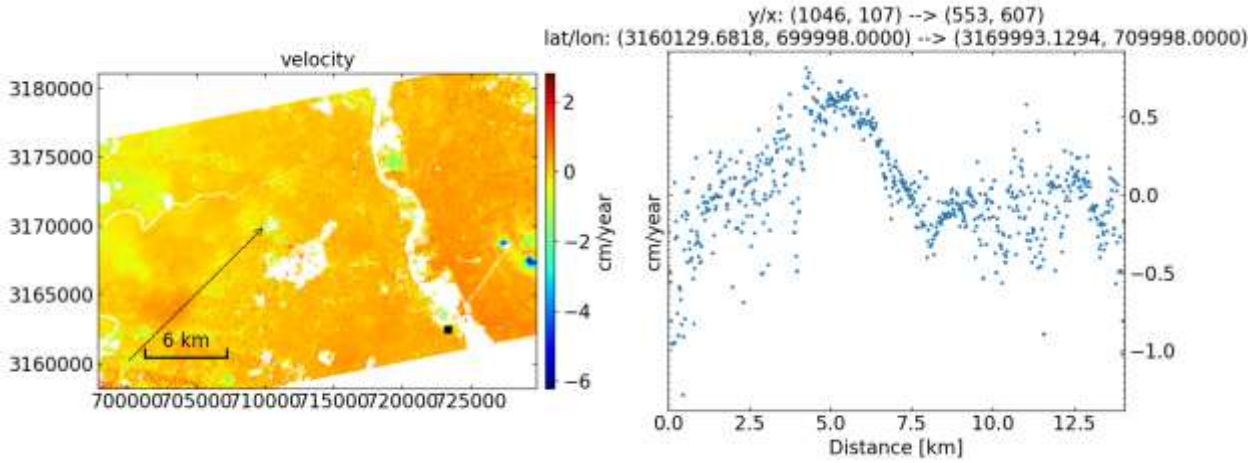


9) Plot a Transect

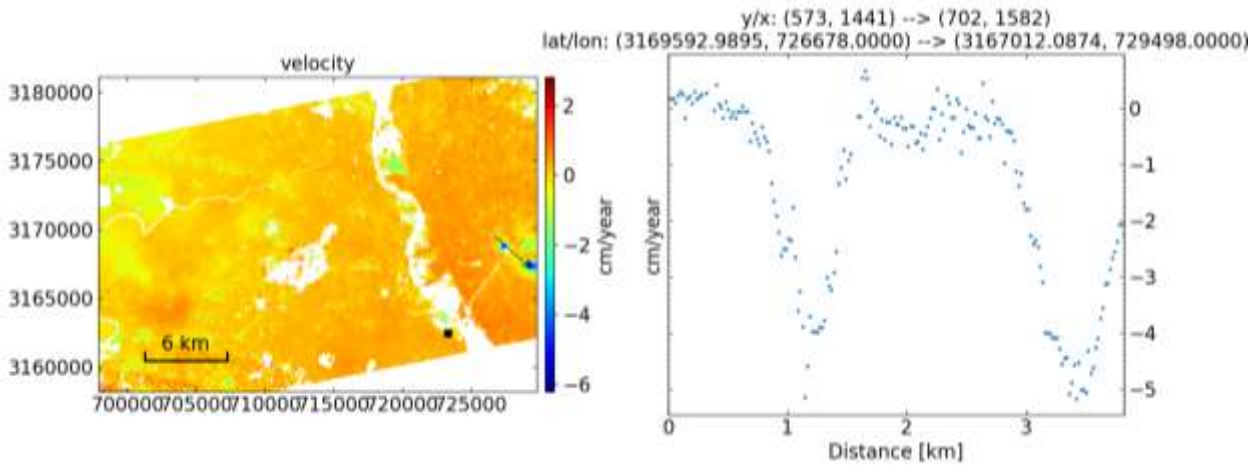
```
This can also be done using two days
## In jupyter-notebook
%matplotlib inline
scp_args = './output/velocity.h5 --start-lalo 3160123 700000 --end-lalo
3170000 710000 --noverbose '
opt = './output/velocity.h5 --start-lalo 3160123 700000 --end-lalo 3170000
710000 --noverbose'
plot_transection.main(scp_args.split())
```




```
## In Terminal window
plot_transection.py ./output/velocity.h5 --start-lalo 3160123 700000 --end-
lalo 3170000 710000
```



Now on the screen, click two points to draw a new profile, and the corresponding graph will be plotted.



12.3.8 Coupling of Ascending and Descending Results

If geocoded displacement products from ascending and descending paths are available, *asc_desc2horz_vert.py* script can be called to project the LOS components to Horizontal and vertical components

12.3.9 Export the Outputs into other formats

MintPy provides tools to export the products in HDF5 format to other formats including ...

- `!save_gbis.py -h    # MATLAB files (GBIS)`
- `!save_qgis.py -h    # GIS Shapefiles`
- `!save_gdal.py -h    # Conversion of Files into Geocoordinates (Geotiff)`
- `!save_gmt.py -h     # Conversion of Files into Geocoordinates (GMT)`
- `!save_kite.py -h    # Kite/Pyrocko file conversion`
- `!save_roipac.py     # exporting into ROI_PAC files`
- `!save_hdfeos5.py    # exporting into HDF-EOS5 files`

```
# Export as kmz file for Google Earth

!smallbaselineApp.py --dir ./output ./output/mintpy_config.txt --dostep
google_earth       # OR
!save_kmz.py ./output/velocity.h5 -c jet_r --ref-size 5 -o
./output/velocity.kmz

# Time-Series export
!save_kmz_timeseries.py ./output/timeseries_SET_tropHgt_ramp_demErr.h5 -c
jet_r -o ./output/Displacement.kmz            # OR
!save_kmz_timeseries.py ./output/timeseries_SET_tropHgt_ramp_demErr.h5 --vel
./output/velocity.h5 --tcoh ./output/temporalCoherence.h5 --mask
./output/maskTempCoh.h5 -c jet_r -o ./Maps/DisplacementTS.kmz

# Exporting as a shapefile
!save_qgis.py ./output/timeseries_SET_tropHgt_ramp_demErr.h5 -g
./output/inputs/geometryGeo.h5 -B LatMin LatMax LonMin LonMax -o
./output/TS_Shapefile.shp
```

13. Conclusion

In this tutorial, a multi-temporal technique of space-borne radar remote sensing i.e. SBAS-InSAR was used to detect, measure, and continuously spatiotemporal monitoring of land deformation of Delhi city with an accuracy of a few cm order. The entire analysis is carried out in the completely free and open source computational environment. The solid earth tides, tropospheric propagation delay, orbital ramps, and topographic phase resideue corrected LOS displacement time-series were estimated and visualized. The negative values of displacement, and velocity are the indicator of subsidence and positive values represent land upliftment. The negative values of acceleration in the city indicate that these areas are still showing movement and are riskier in the near future.

The computation InSAR processing in this tutorial also provide the provisions to plot various parameters under study, coupling of ascending and descending deformation outputs, and export the processed data in many other formats.

Relevant References

[1] H. Fattahi, P. Agram, and M. Simons, "A Network-Based Enhanced Spectral Diversity Approach for TOPS Time-Series Analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 2, pp. 777–786, Feb. 2017, doi: 10.1109/TGRS.2016.2614925.

[2] P. Berardino, G. Fornaro, R. Lanari, and E. Sansosti, "A new algorithm for surface deformation monitoring based on small baseline differential SAR interferograms," *IEEE Trans. Geosci. Remote Sens.*, vol. 40, no. 11, pp. 2375–2383, Nov. 2002, doi: 10.1109/TGRS.2002.803792.

[3] H. A. Zebker, P. A. Rosen, and S. Hensley, "Atmospheric effects in interferometric synthetic aperture radar surface deformation and topographic maps," *J. Geophys. Res. Solid Earth*, vol. 102, no. B4, pp. 7547–7563, Apr. 1997, doi: 10.1029/96JB03804.

[4] H. Fattahi and F. Amelung, "DEM Error Correction in InSAR Time Series," *Geosci. Remote Sens. IEEE Trans. On*, vol. 51, no. 7, pp. 4249–4259, Jun. 2013, doi: 10.1109/TGRS.2012.2227761.

[5] H. Fattahi and F. Amelung, "InSAR bias and uncertainty due to the systematic and stochastic tropospheric delay," *J. Geophys. Res. Solid Earth*, vol. 120, no. 12, pp. 8758–8773, Dec. 2015, doi: 10.1002/2015JB012419.

[6] H. Shankar, D. Singh, and P. Chauhan, "Investigation of groundwater induced land subsidence in Ludhiana City using InSAR and Sentinel-1 data," *Quat. Sci. Adv.*, vol. 13, p. 100151, Jan. 2024, doi: 10.1016/j.qsa.2023.100151.

[7] H. Shankar, D. Singh, and P. Chauhan, "Landslide deformation and temporal prediction of slope failure in Himalayan terrain using PSInSAR and Sentinel-1 data," *Adv. Space Res.*, vol. 70, no. 12, pp. 3917–3931, Dec. 2022, doi: 10.1016/j.asr.2022.04.062.

[8] H. Shankar *et al.*, "Multi-temporal InSAR and Sentinel-1 for assessing land surface movement of Joshimath town, India," *Geomat. Nat. Hazards Risk*, vol. 14, no. 1, p. 2253972, Dec. 2023, doi: 10.1080/19475705.2023.2253972.

[9] Z. Yunjun, H. Fattahi, and F. Amelung, "Small baseline InSAR time series analysis: Unwrapping error correction and noise reduction," *Comput. Geosci.*, vol. 133, p. 104331, Dec. 2019, doi: 10.1016/j.cageo.2019.104331.